How to Copy-Protect All Puncturable Functionalities Without Conjectures: A Unified Solution to Quantum Protection

Alper Çakan
Carnegie Mellon University
alpercakan98@gmail.com

Vipul Goyal
NTT Research & Carnegie Mellon University
vipul@vipulgoyal.org

Abstract

Quantum copy-protection (Aaronson, CCC'09) is the problem of encoding a functionality/key into a quantum state to achieve an anti-piracy security notion that guarantees that the key cannot be split into two keys that both still work. Most works so far has focused on constructing copy-protection for specific functionalities. The only exceptions are the work of Aaronson, Liu, Liu, Zhandry, Zhang (CRYPTO'21) and Ananth and Behera (CRYPTO'24). The former constructs copy-protection for all functionalities in the *classical oracle model* and the latter constructs copy-protection for all circuits that can be punctured at a uniformly random point with negligible security, assuming a new unproven conjecture about simultaneous extraction from entangled quantum adversaries, on top of assuming subexponentially-secure indistinguishability obfuscation (iO) and hardness of Learning with Errors (LWE).

In this work, we show that the construction of Aaronson et al (CRYPTO'21), when the oracles are instantiated with iO, satisfies copy-protection security in the plain model for all *cryptographically puncturable* functionalities (instead of only puncturable circuits) with arbitrary success threshold (e.g. we get CPA-style security rather than unpredictability for encryption schemes), **without any unproven conjectures**, assuming only subexponentially secure iO and one-way functions (we do not assume LWE). Thus, our work resolves the five-year-old open question of Aaronson et al, and further, our work encompasses/supersedes and significantly improves upon all existing plain-model copy-protection results.

Since puncturability has a long history of being studied in cryptography, our result immediately allows us to obtain copy-protection schemes for a large set of advanced functionalities for which no previous copy-protection scheme existed. Further, even for any functionality F that has not already been considered, through our result, constructing copy-protection for F essentially becomes a classical cryptographer's problem.

Going further, we show that our scheme also satisfies *secure leasing* (Ananth and La Placa, EUROCRYPT'21), *unbounded/LOCC leakage-resilience* and *intrusion-detection* security (Çakan, Goyal, Liu-Zhang, Ribeiro, TCC'24), giving a unified solution to the problem of *quantum protection*.

Contents

1 Introduction

Starting with the seminal work of Wiesner [Wie83], a long line of research has shown that quantum information can be extremely useful in cryptography. The first applications of quantum information for cryptography were *quantum money* [Wie83] and *quantum key-distribution* [BB84]. Interestingly, the first two applications also exemplify the two main areas where quantum information helps cryptography: Achieving a classically impossible task[1] and weakening the cryptographic assumptions for a task that is achievable classically (albeit with stronger assumptions)[2]. We name the first area *quantum protection*[3], which we define in more detail as the problem of encoding a piece of information (such as a ciphertext, signature, or a decryption key) into a quantum state to obtain a security guarantee that is impossible to obtain in a classical world. This notion can further be divided into four main categories: *copy-protection* ([Aar09]), *secure leasing* ([AL21]), *unbounded/LOCC leakage-resilience* ([ÇGLZR24]) and *intrusion-detection* ([ÇGLZR24]). In this work, our focus is quantum protection, treated in a unified way.

Most works so far (e.g. [CLLZ21, LLQZ22, ÇG24a, ÇGLZR24]) has focused on constructing quantum protection schemes for specific functionalities (e.g. public-key encryption), with the exception of the work of Aaronson et al ([ALL+21]) and the recent work of Ananth and Behera ([AB24]). [ALL+21] shows how to copy-protect all functionalities using classical structured oracles. [AB24] shows how to copy-protect circuits that can be negligibly-securely punctured at a *uniformly* random point, assuming a **new unproven conjecture** about simultaneous extraction from entangled quantum adversaries, on top of assuming indistinguishability obfuscation (iO) and hardness of Learning with Errors (LWE). Thus, in terms of copy-protecting general functionalities, the current state of knowledge leaves a lot to be desired. Thus, we ask the following question.

> *Can we copy-protect general classes of functionalities in the* plain model*, without any* unproven conjectures*?*

1.1 Our Results

In this work, we answer this question affirmatively.

Theorem 1 (Corollary 1, informal)**.** *For any functionality* F *that satisfies* cryptographical puncturing *security (Definition 5), there exists a* plug-and-play *quantum protection scheme for* F *in the plain model that satisfies at the same time* copy-protection*,* publicly-verifiable classical-certificate secure leasing*,* LOCC leakage-resilience *and* intrusion-detection *security; assuming subexponentially secure iO and one-way functions.*

We give our construction in Section 5. We prove copy-protection security in Section 8, secure-leasing security in Section 6 and LOCC leakage-resilience security in Section 7. We suggest first reading Section 6 or Section 7, and then reading Section 8, since Section 6 and Section 7 are significantly easier (due to not having to deal with extracting from two entangled adversaries) but at the same time they share some techniques with Section 8, thus they can serve as a warm-up for Section 8.

[1] Cryptographic money that can be transferred without the bank's involvement is impossible classically since classical information can be copied/cloned freely.

[2] For example, key distribution with computational security can be achieved using cryptographical assumptions.

[3] While some work refer to this area as *unclonable cryptography*, some recent work ([ÇGLZR24, ÇGS25]) has shown that we can get classically-impossible security guarantees even using publicly efficiently-clonable states, thus, we see quantum protection as a more accurate name.

Comparison to Prior Work Our work encompasses and significantly improves upon all existing plain-model copy-protection results. For example, our work immediately gives copy-protection for public-key encryption, pseudorandom functions, signatures and functional encryption, superseding the works of [CLLZ21, LLQZ22, ÇG24a] who gave specific copy-protection constructions for these specific primitives[4]. In terms of general functionalities, even the assumption of the unproven conjecture aside, [AB24] only shows copy-protection for puncturable circuits with uniform challenge distributions (which does not capture e.g. encryption schemes where the challenge is a ciphertext, not a uniform string) that satisfy negligible security (for example it does not capture CPA-style games where the baseline security is 1/2).

Further, our work only uses one-way functions and iO, and does not use the structured assumption of hardness of Learning with Errors (LWE). This is in contrast to all previous plain-model constructions (e.g. [CLLZ21, LLQZ22, ÇG24a, AB24]) which do assume LWE, except for the beautiful prior work of [KY25]. [KY25] shows copy-protection assuming one-way functions and iO, but only for public-key encryption. Thus, our work encompasses and improves upon their construction too[5].

Applications Since puncturability has a long history of being studied in cryptography, our result immediately allows us to obtain quantum protection schemes for a large set of advanced functionalities for which no previous protection scheme existed. For example, by tapping into the vast amount of classical cryptography literature, we obtain the following (non-exhaustive) list of new results.

Theorem 2 (Informal)**.** *Assuming iO and one-way functions, there exist plain model quantum protection schemes for*

- *decryption keys of a publicly deniable encryption scheme ([SW14]),*
- *functional keys of a multi-input functional encryption scheme ([GGG+14]),*
- *functional keys of a functional encryption scheme for randomized functionalities ([GJKS15]),*
- *signing keys of a rerandomizable signature scheme ([CGY24], additionally assuming LWE),*
- *keys of a key-homomorphic PRF scheme ([BV15], additionally assuming LWE),*
- *functional keys of a functional encryption scheme for RAM programs ([ACFQ22, LMW23], additionally assuming RingLWE),*
- *keys of an invertible PRF scheme ([BKW17], addtionally assuming LWE),*
- *functional keys of a function-hiding functional encryption scheme ([GGH+13]),*
- *master secret key of an identity-based encryption scheme ([GGH+13]),*

[4]Though [LLQZ22] and [ÇG24a] in particular also achieve bounded collusion-resistant and fully collusion-resistant copy-protection, respectively, for specific functionalities. In this work, we do not consider collusiion-resistant copy-protection.

[5]However, they additionally show copy-protection for public-key encryption with various correlated challenge distributions, which we do not consider in this work.

Here, a quantum protection scheme for $\mathcal{F}$ means a single scheme that satisfies all of the copy-protection, LOCC leakage-resilience, intrusion detection, and secure leasing security notions (each security definition is specialized to the security notion for that primitive[6]).

Further, even for any functionality F that has not already been considered, through our result, constructing quantum protection for F essentially becomes a classical cryptographer's problem.

***Plug-and-Play* Schemes** Finally, we note that all our schemes are *plug-and-play*, which means that it consists of two stages: a first stage that outputs a quantum state and public parameters that does not depend on the functionality at all, and a second stage that receives the public parameters and the circuit of the functionality to be protected and outputs only a classical string. This modular system can even allow copy-protection to be build on top of, for example, quantum money schemes that are in circulation.

1.2 Other Related Work

[CG24b] and [BBV24] define the notion of *quantum state indistinguishability obfuscation* (qsiO) and show that it gives a concrete copy-protection scheme for any functionality for which a secure copy-protection scheme exists. [BBV24] shows how to construct qsiO using classical oracles. However, these works have no plain model results for copy-protection.

2 Preliminaries

2.1 Notation

When S is a set, we write $x \leftarrow S$ to mean that x is sampled uniformly at random from S. When $\mathcal{D}$ is a distribution, we write $x \leftarrow \mathcal{D}$ to mean that x is sampled from $\mathcal{D}$. Finally, we write $x \leftarrow \mathcal{B}(\mathsf{R})$ or $\mathcal{B}(a)$ to mean that x is set to a sample as output by the quantum or classical randomized algorithm $\mathcal{B}$ run on R or a. We use similar notation for quantum registers, e.g., $\mathsf{R} \leftarrow \mathcal{B}(a)$.

We write QPT to mean a stateful[7] *quantum polynomial time*. We write R to mean a quantum register, which keeps a quantum state that will evolve when the register is acted on, and it can be entangled with other registers. We write $\mathsf{R} \leftarrow \rho$ to mean that the register R is initialized with a sample from the quantum distribution (i.e. mixed state) ρ. We refer the reader to [NC10] for a preliminary on quantum information and computation.

Unless otherwise specified, all of our cryptographical assumptions are implicitly post-quantum, e.g., *one-way functions* means *post-quantum secure one-way functions*.

2.2 Cryptography

2.2.1 Puncturable Pseudorandom Functions

In this section, we introduce puncturable pseudorandom functions.

[6]For example, for securely-leasable decryption keys of a publicly deniable encryption scheme, the security requirement is that once an adversary's decryption key is successfully returned, a challenge ciphertext is deniable now even to the adversary. Note that this is impossible classically, since it is not possible to produce a fake ciphertext randomness (opening the ciphertext to a different message) against an adversary who has the secret key, simply due to correctness of the scheme.

[7]Unless otherwise specified

Definition 1 ([SW14])**.** *A puncturable pseudorandom function (PRF) is a family of functions* $\{F : \{0,1\}^{c(\lambda)} \times \{0,1\}^{m(\lambda)} \to \{0,1\}^{n(\lambda)}\}_{\lambda \in \mathbb{N}^+}$ *with the following associated efficient algorithms.*

- $\mathsf{PRF.Setup}(1^\lambda)$: *Takes in a security parameter and outputs a key in* $\{0,1\}^{c(\lambda)}$.
- $\mathsf{PRF.Eval}(K, x)$: *Takes in a key and an input, outputs an evaluation of the PRF.*
- $\mathsf{PRF.Puncture}(K, S)$: *Takes as input a key and a set* $S \subseteq \{0,1\}^{m(\lambda)}$*, outputs a punctured key.*

We require the following.

Correctness. *For all efficient distributions* $\mathcal{D}(1^\lambda)$ *over the power set* $2^{\{0,1\}^{m(\lambda)}}$*, we require*

$$\Pr\left[\forall x \notin S \quad \mathsf{PRF.Eval}(K_S, x) = F(K, x) : \begin{array}{c} S \leftarrow \mathcal{D}(1^\lambda) \\ K \leftarrow \mathsf{PRF.Setup}(1^\lambda) \\ K_S \leftarrow \mathsf{PRF.Puncture}(K, S) \end{array}\right] = 1.$$

Puncturing Security *We require that any stateful QPT adversary* $\mathcal{A}$ *wins the following game with probability at most* $1/2 + \mathsf{negl}(\lambda)$.

1. $\mathcal{A}$ *outputs a set* S.
2. *The challenger samples* $K \leftarrow \mathsf{PRF.Setup}(1^\lambda)$ *and* $K_S \leftarrow \mathsf{PRF.Puncture}(K, S)$
3. *The challenger samples* $b \leftarrow \{0,1\}$*. If* $b = 0$*, the challenger submits* $K_S, \{F(K,x)\}_{x \in S}$ *to the adversary. Otherwise, it submits* $K_S, \{y_s\}_{s \in S}$ *to the adversary where* $y_s \leftarrow \{0,1\}^{n(\lambda)}$ *for all* $s \in S$.
4. *The adversary outputs a guess* b' *and we say that the adversary has won if* $b' = b$.

Theorem 3 ([SW14, Zha12])**.** *Let* $n(\cdot), m(\cdot), e(\lambda), k(\lambda)$ *be efficiently computable functions.*

- *If (post-quantum) one-way functions exist, then there exists a (post-quantum) puncturable PRF with input space* $\{0,1\}^{m(\lambda)}$ *and output space* $\{0,1\}^{n(\lambda)}$.
- *If we assume subexponentially-secure (post-quantum) one-way functions exist, then for any* $c > 0$*, there exists a (post-quantum)* $2^{-\lambda^c}$*-secure*[8] *puncturable PRF against subexponential time adversaries with input space* $\{0,1\}^{m(\lambda)}$ *and output space* $\{0,1\}^{n(\lambda)}$.

2.2.2 Indistinguishability Obfuscation

In this section, we introduce indistinguishability obfuscation.

Definition 2. *An indistinguishability obfuscation scheme* $i\mathcal{O}$ *for a class of circuits* $\mathcal{C} = \{\mathcal{C}_\lambda\}_\lambda$ *satisfies the following.*

Correctness. *For all* $\lambda, C \in \mathcal{C}_\lambda$ *and inputs* x, $\Pr\left[\tilde{C}(x) = C(x) : \tilde{C} \leftarrow i\mathcal{O}(1^\lambda, C)\right] = 1$.

[8] While the original results are for negligible security against polynomial time adversaries, it is easy to see that they carry over to subexponential security. Further, by scaling the security parameter by a polynomial and simple input/output conversions, subexponentially secure (for any exponent c') one-way functions is sufficient to construct for any c a puncturable PRF that is $2^{-\lambda^c}$-secure.

Security. Let $\mathcal{B}$ be any QPT algorithm that outputs two circuits $C_0, C_1 \in \mathcal{C}$ of the same size, along with auxiliary information, such that $\Pr\left[\forall x\ C_0(x) = C_1(x) : (C_0, C_1, \mathsf{R}_{\mathsf{aux}}) \leftarrow \mathcal{B}(1^\lambda)\right] \geq 1 - \mathsf{negl}(\lambda)$. Then, for any QPT adversary $\mathcal{A}$,

$$\left| \Pr\left[\mathcal{A}(i\mathcal{O}(1^\lambda, C_0), \mathsf{R}aux) = 1 : (C_0, C_1, \mathsf{R}_{\mathsf{aux}}) \leftarrow \mathcal{B}(1^\lambda)\right] - \Pr\left[\mathcal{A}(i\mathcal{O}(1^\lambda, C_1), \mathsf{R}_{\mathsf{aux}}) = 1 : (C_0, C_1, \mathsf{R}_{\mathsf{aux}}) \leftarrow \mathcal{B}(1^\lambda)\right] \right| \leq \mathsf{negl}(\lambda).$$

2.2.3 Asymmetrically Constrainable Encapsulation

We recall the definition of *Asymmetrically Constrainable Encapsulation*, mostly verbatim from [CHJV15]. Let $\mathcal{M} = \{0,1\}^n$ denote the message space, where $n = \mathrm{poly}(\lambda)$ for security parameter λ. An (ℓ, s)-asymmetrically constrained encapsulation scheme consists of five algorithms:

- **Setup:** $\mathsf{Setup}(1^\lambda)$ outputs a secret key SK.
- **Key Generation:** Let $S \subseteq \mathcal{M}$ be a set decidable by a circuit C_S of size at most s such that $C_S(m) = \mathsf{TRUE}$ if and only if $m \in S$. Then:
  - $\mathsf{GenEK}(\mathsf{SK}, C_S)$ outputs an encapsulation key EK_S.
  - $\mathsf{GenDK}(\mathsf{SK}, C_S)$ outputs a decapsulation key DK_S.
- **Encapsulation:** $\mathsf{Enc}(\mathsf{EK}', m)$ is a deterministic algorithm that takes a possibly constrained encapsulation key EK' and a message $m \in \mathcal{M}$ and outputs a ciphertext c or $\perp$.
- **Decapsulation:** $\mathsf{Dec}(\mathsf{DK}', c)$ is a deterministic algorithm that takes a possibly constrained decapsulation key DK' and a ciphertext c and outputs a message $m \in \mathcal{M}$ or $\perp$.

Correctness:

1. **Decapsulation correctness:** For all admissible S, S' and $m \notin S \cap S'$,
$$\Pr\left[\mathsf{SK} \leftarrow \mathsf{Setup}(1^\lambda) : \mathsf{Dec}(\mathsf{GenDK}(\mathsf{SK}, C_S), \mathsf{Enc}(\mathsf{GenEK}(\mathsf{SK}, C_{S'}), m)) = m\right] = 1$$
2. **Equivalence of Constrained Encapsulation** For all S and $m \notin S$,
$$\Pr\left[\mathsf{SK} \leftarrow \mathsf{Setup}(1^\lambda) : \mathsf{Enc}(\mathsf{GenEK}(\mathsf{SK}, C_S), m) = \mathsf{Enc}(\mathsf{GenEK}(\mathsf{SK}, \emptyset), m)\right] = 1$$
3. **Constrained decapsulation safety:** For all S and ciphertexts c and messages $m \in S$,
$$\Pr[m = \mathsf{Dec}(\mathsf{GenDK}(\mathsf{SK}, C_S), c)] = 0$$
4. **Constrained decapsulation equivalence:** For all S and all ciphertexts c,
$$\Pr\left[\begin{array}{l} \mathsf{SK} \leftarrow \mathsf{Setup}(1^\lambda), \\ \mathsf{DK} \leftarrow \mathsf{GenDK}(\mathsf{SK}, \emptyset), \\ \mathsf{DK}_S \leftarrow \mathsf{GenDK}(\mathsf{SK}, C_S), \\ m \leftarrow \mathsf{Dec}(\mathsf{DK}, c), \\ m' \leftarrow \mathsf{Dec}(\mathsf{DK}_S, c) : \\ \text{either } m \in S \text{ or } m' = m \end{array}\right] = 1$$

Security of Constrained Decapsulation (*Puncture Hiding Security*): For all QPT adversaries A that interact as follows:

- A chooses $S_0 \subseteq S_1 \subseteq U \subseteq \mathcal{M}$ and provides their circuit representations.
- A also provides messages $m_1, \ldots, m_\ell$ such that $m_i \notin S_1 \setminus S_0$.
- The challenger samples $\mathsf{SK} \leftarrow \mathsf{Setup}(1^\lambda)$, and computes:

$$\mathsf{EK}_U \leftarrow \mathsf{GenEK}(\mathsf{SK}, C_U), \quad \mathsf{DK}_{S_b} \leftarrow \mathsf{GenDK}(\mathsf{SK}, C_{S_b})$$

  for a random bit $b \in \{0,1\}$, and sends EK_U, DK_{S_b}, and $\mathsf{Enc}(\mathsf{EK}, m_i)$ to A.
- A outputs a guess b'.

Then:

$$\Pr\left[b' = b\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

Pseudorandom Ciphertexts Let $\ell \in \mathbb{N}$.

- A selects two sets $S, U \subseteq \mathcal{M}$ and provides circuits C_S, C_U.
- A submits ℓ messages:

$$(m_1^0), \ldots, (m_\ell^0) \quad \text{such that } m_i^0 \in S \cap U$$

- The challenger samples $\mathsf{SK} \leftarrow \mathsf{Setup}(1^\lambda)$, computes:

$$\mathsf{EK}_U \leftarrow \mathsf{GenEK}(\mathsf{SK}, C_U)$$
$$\mathsf{DK}_S \leftarrow \mathsf{GenDK}(\mathsf{SK}, C_S)$$

- The challenger samples $b \in \{0,1\}$ and computes ciphertexts:

$$c_i = \mathsf{Enc}(\mathsf{EK}, m_i^0) \quad \text{for } i = 1, \ldots, \ell$$

  if $b = 0$ and otherwise samples c_i uniformly at random.
- The adversary receives $(\mathsf{EK}_U, \mathsf{DK}_S, c_1, \ldots, c_\ell)$ and outputs a guess b'.

Then, for all QPT adversaries A,

$$\Pr\left[b' = b\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

Theorem 4 ([CHJV15], implicit)**.** *Assuming subexponentially secure iO and one-way functions, subexponentially secure* ACE *with pseudorandom ciphertexts exists.*

2.3 Quantum Information

Lemma 1 (Almost As Good As New Lemma [Aar16], verbatim)**.** *Let ρ be a mixed state acting on $\mathbb{C}^d$. Let U be a unitary and $(\Pi_0, \Pi_1 = I - \Pi_0)$ be projectors all acting on $\mathbb{C}^d \otimes \mathbb{C}^{d'}$. We interpret (U, Π_0, Π_1) as a measurement performed by appending an ancillary system of dimension d' in the state $|0\rangle\langle 0|$, applying U and then performing the projective measurement Π_0, Π_1 on the larger system. Assuming that the outcome corresponding to Π_0 has probability $1 - \epsilon$, we have*

$$\left\|\rho - \rho'\right\|_{Tr} \leq \sqrt{\epsilon}$$

where ρ' is the state after performing the measurement, undoing the unitary U and tracing out the ancillary system.

Theorem 5 ([ÇG24a])**.** *Let ρ be a bipartite state and $\Lambda = \{\Pi_1, \dots\}, \Lambda' = \{\Pi'_1, \dots\}$ be two projective measurements over each of these registers, respectively. Suppose*

$$\mathrm{Tr}\{\Pi_1 \otimes \Pi'_1 \rho\} \geq 1 - \epsilon.$$

Let $M = \{M_i\}_{i \in \mathcal{I}}$ be a general measurement over the first register and fix any $i \in \mathcal{I}$. Let τ denote the post-measurement state of the second register after applying the measurement M on the first register of ρ and conditioned on obtaining outcome i. Let p_i denote probability of outcome i, that is $p_i = \mathrm{Tr}\Big\{(M_i \otimes I)\rho(M_i^\dagger \otimes I)\Big\}$. Then,

$$\mathrm{Tr}\{\Pi'_1 \tau\} \geq 1 - \frac{3\sqrt{\epsilon}}{2p_i}.$$

Theorem 6 (Implementation Independence of Measurements on Bipartite States ([ÇG24a]))**.** *Let $\Lambda = \{M_i\}_{i \in \mathcal{I}}, \Lambda' = \{E_i\}_{i \in \mathcal{I}}$ be two general measurements whose POVMs are equivalent, that is, $M_i^\dagger M_i = E_i^\dagger E_i$ for all $i \in \mathcal{I}$.*

Let ρ be any bipartite state whose first register has the appropriate dimension for Λ, Λ'. Then, the post-measurement state of the second register conditioned on any outcome $i \in \mathcal{I}$ is the same when either Λ or Λ' is applied to the first register of ρ. That is,

$$(\mathrm{Tr} \otimes I)\frac{(M_i \otimes I)\rho(M_i^\dagger \otimes I)}{\mathrm{Tr}\Big\{(M_i \otimes I)\rho(M_i^\dagger \otimes I)\Big\}} = (\mathrm{Tr} \otimes I)\frac{(E_i \otimes I)\rho(E_i^\dagger \otimes I)}{\mathrm{Tr}\Big\{(E_i \otimes I)\rho(E_i^\dagger \otimes I)\Big\}}$$

3 Definitional Work

3.1 Puncturable Cryptography

We first define a template that captures most cryptographic functionalities along with their security games.

Definition 3 (Cryptographic Functionality)**.** *A cryptographic functionality $\mathcal{F}$ is defined by a mapping Φ and a tuple of efficient algorithms $(\mathsf{Eval}, \mathsf{Chal}, \mathcal{D}, \mathsf{Ver})$, defined as follows.*

- $\Phi : \mathcal{S} \times \mathcal{X} \to \mathcal{Y}$ *represents the functionality in terms of the input-output behavior,*
- Eval *is a (possibly quantum) algorithm that receives as input $s \in \mathcal{S}$, some $x \in \mathcal{X}$ and outputs some $y \in \mathcal{Y}$,*

- Chal *is a (possibly quantum) interactive algorithm that represents the setup of the security game such that at the end of the interaction with an adversary, it outputs a secret* $s \in \mathcal{S}$ *and challenge parameters*[9] *cp,*
- $\mathcal{D}$ *is a classical input-output (possibly quantum) sampler that receives as input cp and s, and outputs a challenge ch (which will be sent to the adversary) and secret parameters sp,*
- Ver *is a (possibly quantum) verifier that receives as input the secret parameters*[10] *sp and an alleged answer* $y \in \mathcal{Y}$*, and it outputs* TRUE *or* FALSE*.*

We define correctness and security as below.

Correctness: *We require that* $\mathsf{Eval}(s, x)$ *outputs* $\Phi(s, x)$.[11]*.*

$p^{\mathcal{F}}_{triv}$-Security: *For any QPT adversary* $\mathcal{A}$*, we have* $\Pr\left[\mathsf{SecurityGame}_{\mathcal{A}}(1^\lambda) = 1\right] \leq p^{\mathcal{F}}_{triv}(\lambda) + \mathsf{negl}(\lambda)$ *where the security game* $\mathsf{SecurityGame}$ *is defined as follows*[12]*.*

$\mathsf{SecurityGame}_{\mathcal{A}}(1^\lambda)$

1. $\mathsf{Chal}(1^\lambda)$ *and* $\mathcal{A}(1^\lambda)$ *interact*[13]*,* Chal *outputs s and cp.*
2. *The challenger runs* $ch, sp \leftarrow \mathcal{D}(cp, s)$*.*
3. *The adversary* $\mathcal{A}$ *receives ch and outputs y.*
4. *The challenger outputs* 1 *if* $\mathsf{Ver}(sp, y) = \mathsf{TRUE}$*, otherwise outputs* 0*.*

For our quantum protection applications, we will also require that the functionality satisfies a high min-entropy condition for its challenges. It is easy to see that for functionalities that do not satisfy this condition, it is impossible to obtain a scheme that satisfies any of the quantum protection notions, even in the oracle model, since the adversary can simply guess the challenge and answer it while it has the key. Thus, when we say functionality, we will implicitly refer to ones that satisfy this condition.

Definition 4 (Unpredictable Challenge Cryptographic Functionality)**.** *A cryptographic functionality is said to be a unpredictable challenge functionality if there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\Pr\left[\forall ch' \ \Pr\left[ch = ch' : ch \leftarrow \mathcal{D}(cp, s)\right] \leq \mathsf{negl}(\lambda) : s, cp \leftarrow \langle \mathsf{Chal}(1^\lambda), \mathcal{A}(1^\lambda)\rangle\right] \geq 1 - \mathsf{negl}(\lambda)$$

.

We now define the notion of cryptographically puncturable functionalities.

[9] This represents the information needed to sample challenge. For example, this could be the challenge messages m_0, m_1 in a CPA security game for encryption.

[10] The secret parameters here represent the information that is needed to verify adversary's answer. For example, in a CPA security game, this will be the challenge bit b where the challenge ciphertext ct is an encryption of m_b.

[11] A more precise definition of correctness will depend on the application. For example, we might require that for all (or for most) $x \in \mathcal{X}$, Eval outputs the correct value with overwhelming probability (or with probability 1).

[12] Here, $p^{\mathcal{F}}_{triv}$ denotes the baseline/trivial success probability. For example, for a chosen plaintext security game, we will have $p^{\mathcal{F}}_{triv} = 1/2$.

[13] During this interaction, $\mathcal{A}$ may learn some information related to the secret s. For example, in a public-key encryption security game, $\mathcal{A}$ will learn the public-key

Definition 5 (Cryptographically Puncturable Cryptographic Functionality)**.** *A cryptographically puncturable cryptographic functionality is a cryptographic functionality* $\mathcal{F} = (\Phi, \mathsf{Eval}, \mathsf{Chal}, \mathcal{D}, \mathsf{Ver})$ *with the following additions.*

- *The secret domain is simply the set of circuits* C *of a fixed size,*
- Eval *simply executes the circuit* C *on the input* x*,*
- *There is an additional efficient (possibly quantum algorithm)* Punc *that receives as input a circuit* C *and an input* x*, outputs another circuit* C_{punc}*.*
- *There is an additional efficient (possibly quantum algorithm)* $\mathcal{D}_{\mathsf{inp}}$ *that receives as input* cp *and* C*, and outputs* $x \in \mathcal{X}$
- *There is an additional efficient (possibly quantum algorithm)* $\mathcal{D}_{\mathsf{chal}}$ *that receives as input* cp*,* C *and* x*, and outputs* $x \in \mathcal{X}$*.*
- $\mathcal{D}$ *is simply the following concatenation of* $\mathcal{D}_{\mathsf{inp}}$ *and* $\mathcal{D}_{\mathsf{chal}}$*: It first runs* $x \leftarrow \mathcal{D}_{\mathsf{inp}}$ *and then* $ch, sp \leftarrow \mathcal{D}_{\mathsf{chal}}(cp, C, x)$*, and finally outputs* $ch, (x, sp')$*.*

Punctured Correctness: *We require that* $C\{x\}(x') = C(x)$ *for all* $x' \neq x$*.*

Puncturing $p^{\mathcal{F}}_{triv}$**-Security:** *For any QPT adversary* $\mathcal{A}$*, we have* $\Pr\left[\mathsf{PuncturingGame}_{\mathcal{A}}(1^\lambda) = 1\right] \leq p^{\mathcal{F}}_{triv}(\lambda) + \mathsf{negl}(\lambda)$ *where the security game* $\mathsf{PuncturingGame}$ *is defined as follows.*

$\mathsf{PuncturingGame}_{\mathcal{A}}(1^\lambda)$

1. ***Setup Phase:*** Chal *and* $\mathcal{A}$ *interact,* Chal *outputs* C *and* cp*.*
2. *The challenger runs* $x \leftarrow \mathcal{D}_{\mathsf{inp}}(cp, C)$*.*
3. *The challenger runs* $ch, sp' \leftarrow \mathcal{D}_{\mathsf{chal}}(cp, C, x)$*.*
4. *Set* $sp = (x, sp')$
5. *The challenge runs* $C_{punc} \leftarrow \mathsf{Punc}(C, x)$*.*
6. ***Challenge Phase:*** *The adversary* $\mathcal{A}$ *receives* cp, ch, C_{punc}, x *and outputs* y*.*
7. *The challenger outputs* 1 *if* $\mathsf{Ver}((x, sp'), y) = \mathsf{TRUE}$*, otherwise outputs* 0*.*

Weak Punctured Sampling Correctness: *We require that with overwhelming probability, sampling* $\mathcal{D}(cp, C_{punc})$ *where* $C_{punc} \leftarrow \mathsf{Punc}(C, x)$ *is equivalent to sampling* $\mathcal{D}(cp, C)$*. That is, the punctured keys can be used to resample challenges.*

Pseudorandom puncturing points: *We require that the punctured points* x*, sampled through* $\mathcal{D}_{\mathsf{inp}}$ *are pseudorandom.*

3.2 Quantum Protection Definitions

Let $\mathcal{F} = (\Phi, \mathsf{Eval}, \mathsf{Chal}, \mathcal{D}, \mathsf{Ver})$ be a cryptographic functionality (Definition 3).

We first recall copy-protection security, introduced by [Aar09], which is also referred to as *anti-piracy security* or *unclonability security*.

Definition 6 (Copy-Protection)**.** *A quantum protection scheme* $\mathsf{QuantumProtect}$ *is said to satisfy copy-protection security if for any QPT adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1\mathcal{A}_2)$*, we have* $\Pr\left[\mathsf{AntiPiracy}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \leq p^{\mathcal{F}}_{triv}(\lambda) + \mathsf{negl}(\lambda)$ *where the security game* $\mathsf{AntiPiracy}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)$ *is defined as follows.*

$\underline{\mathsf{AntiPiracy}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)}$

1. *Sample* $pp, \mathsf{R}' \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$.
2. $\mathsf{Chal}(1^\lambda)$ *and* $\mathcal{A}(1^\lambda)$ *interact,* Chal *outputs* C *and* cp.
3. *Sample* $\hat{P} \leftarrow \mathsf{QuantumProtect.Protect}(pp, C)$.
4. *Submit* $\hat{P}, \mathsf{R}', pp$ *to* $\mathcal{A}_0$.
5. *The adversary* $\mathcal{A}_0$ *outputs two registers* $\mathsf{R}_1, \mathsf{R}_2$.
6. *The challenger runs* $ch_1, sp_1 \leftarrow \mathcal{D}(cp, C)$ *and* $ch_2, sp_2 \leftarrow \mathcal{D}(cp, C)$.
7. $\mathcal{A}_1$ *receives* ch_1, sp_1, R_1*, outputs* y_1.
8. $\mathcal{A}_2$ *receives* ch_2, sp_2, R_2*, outputs* y_2.
9. $b_{1,Ver} \leftarrow \mathsf{Ver}(sp_1, y_1)$ *and* $b_{2,Ver} \leftarrow \mathsf{Ver}(sp_2, y_2)$.
10. *The challenger outputs* 1 *if* $b_{1,Ver} = 1 \wedge b_{2,Ver} = 1$*, otherwise outputs* 0.

We also recall strong anti-piracy.

Definition 7 (Strong Anti-Piracy)**.** *Let* γ *be an inverse polynomial. A quantum protection scheme* $\mathsf{QuantumProtect}$ *is said to satisfy strong copy-protection security if for any QPT adversary* $\mathcal{A} = \mathcal{A}_0$*, we have* $\Pr\left[\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \leq \mathsf{negl}(\lambda)$ *where the security game* $\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect}}$ *is defined as follows.*

$\underline{\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)}$

1. *Sample* $pp, \mathsf{R}' \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$.
2. $\mathsf{Chal}(1^\lambda)$ *and* $\mathcal{A}(1^\lambda)$ *interact,* Chal *outputs* C *and* cp.
3. *Sample* $\hat{P} \leftarrow \mathsf{QuantumProtect.Protect}(pp, C)$.
4. *Submit* $\hat{P}, \mathsf{R}', pp$ *to* $\mathcal{A}_0$.
5. *The adversary* $\mathcal{A}_0$ *outputs two registers* $\mathsf{R}_1, \mathsf{R}_2$.
6. *Apply the threshold implementation* $\mathsf{TI}_{(\mathsf{Ver}\circ U, \mathcal{D}(cp,C)), p^{\mathcal{F}}_{triv}+\gamma} \otimes \mathsf{TI}_{(\mathsf{Ver}\circ U, \mathcal{D}(cp,C)), p^{\mathcal{F}}_{triv}+\gamma}$ *to* $\mathsf{R}_1, \mathsf{R}_2$.
7. *The challenger outputs* 1 *if both measurements output* 1*, otherwise outputs* 0.

As shown by [CLLZ21], a scheme that satisfies strong anti-piracy for any inverse polynomial γ also satisfies regular anti-piracy (Definition 6).

We now recall secure leasing security, introduced by [AL21]. This has also been called *secure key leasing* or *revocable* security.

Definition 8 (Secure Leasing)**.** *A quantum protection scheme* $\mathsf{QuantumProtect}$ *is said to satisfy secure leasing security if for any QPT adversary* $\mathcal{A}$*, we have* $\Pr\left[\mathsf{SecureLeasingGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \leq p^{\mathcal{F}}_{triv}(\lambda) + \mathsf{negl}(\lambda)$ *where the security game* $\mathsf{SecureLeasingGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)$ *is defined as follows.*

$\underline{\mathsf{SecureLeasingGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)}$

1. *Sample* $pp, \mathsf{R}' \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$.
2. $\mathsf{Chal}(1^\lambda)$ *and* $\mathcal{A}(1^\lambda)$ *interact,* Chal *outputs* C *and* cp.
3. *Sample* $\hat{P} \leftarrow \mathsf{QuantumProtect.Protect}(pp, C)$.
4. *Submit* $\hat{P}, \mathsf{R}', pp$ *to* $\mathcal{A}$.
5. $\mathcal{A}$ *outputs* $cert$.
6. $b_{cert} \leftarrow \mathsf{QuantumProtect.CertVerify}(pp, cert)$.
7. *The challenger runs* $ch, sp \leftarrow \mathcal{D}(cp, C)$.
8. *Submit* ch *to* $\mathcal{A}$*,* $\mathcal{A}$ *outputs* y.
9. $b_{Ver} \leftarrow \mathsf{Ver}(sp, y)$.
10. *The challenger outputs* 1 *if* $\mathsf{Ver}(sp, y) = \mathsf{TRUE}$ *and* $b_{cert} = \mathsf{TRUE}$*, otherwise outputs* 0.

We recall LOCC (local operations and classical communication) security, introduced by [ÇGLZR24]. The special case where the adversary only obtains a single round of leakage is also called *unbounded leakage-resilience security.*

Definition 9 (LOCC Leakage-Resilience)**.** *A quantum protection scheme* $\mathsf{QuantumProtect}$ *is said to satisfy LOCC leakage-resilience security if for any QPT LOCC leakage adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$*, we have* $\Pr\left[\mathsf{LeakageGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \leq p^{\mathcal{F}}_{triv}(\lambda) + \mathsf{negl}(\lambda)$ *where the security game* $\mathsf{LeakageGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)$ *is defined as follows.*

$\underline{\mathsf{LeakageGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)}$

1. *Sample* $pp, \mathsf{R}' \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$.
2. $\mathsf{Chal}(1^\lambda)$ *and* $\mathcal{A}(1^\lambda)$ *interact,* Chal *outputs* C *and* cp.
3. *Sample* $\hat{P} \leftarrow \mathsf{QuantumProtect.Protect}(pp, C)$.
4. *Submit* $\hat{P}, \mathsf{R}', pp$ *to* $\mathcal{A}_1$.
5. $\mathcal{A}_1$ *and* $\mathcal{A}_2$ *execute their LOCC leakage protocol.*
6. *The challenger runs* $ch, sp \leftarrow \mathcal{D}(cp, C)$.

7. *Submit ch to $\mathcal{A}_2$, $\mathcal{A}_2$ outputs y.*

8. $b_{Ver} \leftarrow \mathsf{Ver}(sp, y)$.

9. *The challenger outputs* 1 *if* $\mathsf{Ver}(sp, y) = \mathsf{TRUE}$ *and* $b_{cert} = \mathsf{TRUE}$*, otherwise outputs* 0.

4 Technical Tools

4.1 Quantum Protection Properties of Coset States

We first give the following lemma, which essentially helps separate the computational and the information-theoretic quantum protection properties of coset states.

Lemma 2. *Consider the following (parametrized) game* $\mathsf{Game}^b_{\mathcal{A}}(1^\lambda)$ *(for* $c \in \{0,1\}$*) between a challenger and an adversary* $\mathcal{A}$*.*

$\underline{\mathsf{Game}^c_{\mathcal{A}}(1^\lambda)}$

1. *The challenger samples a* $\mathbb{F}_2^{d(\lambda)}$*-coset as follows: Sample a subspace* $A \leq \mathbb{F}_2^{d(\lambda)}$ *and two elements* $a_1, a_2 \in \mathbb{F}_2^{d(\lambda)}$ *uniformly at random.*

2. *The challenger initialize the register* R *with the coset state* $|A_{a_1,a_2}\rangle = \sum_{v\in A}(-1)^{\langle v,a_2\rangle}|v + a_1\rangle$.

3. *The challenger samples a superspace* B_1 *of* A *of dimension* $\frac{3d}{4}$*, and a subspace* B_2 *of* A *of dimension* $\frac{d}{4}$ *and elements* $z_1 \leftarrow B_1$*,* $z_2 \leftarrow B_2^\perp$*. The challenger also sets* $t = z_1 + a_1$ *and* $t' = z_2 + a_2$*.*

4. *The challenger samples* $\hat{M} \leftarrow i\mathcal{O}(M^c)$*.*

> $\underline{M^0(b, v)}$
> **Hardcoded:** A, a_1, a_2
>
> 1. *If* $b = 0$*: Check if* $v \in A + a_1$*. If so, output* TRUE*, otherwise output* FALSE*, and terminate.*
> 2. *If* $b = 1$*: Check if* $v \in A^\perp + a_2$*. If so, output* TRUE*, otherwise output* FALSE*.*

> $\underline{M^1(b, v)}$
> **Hardcoded:** t, t', B_1, B_2
>
> 1. *If* $b = 0$*: Check if* $v \in B_1 + t$*. If so, output* TRUE*, otherwise output* FALSE*, and terminate.*
> 2. *If* $b = 1$*: Check if* $v \in B_2^\perp + t'$*. If so, output* TRUE*, otherwise output* FALSE*.*

5. *The challenger sends* $\mathsf{R}, A, B_1, B_2, t, t'$ *to the adversary* $\mathcal{A}$*.*

6. *The adversary outputs a register* $\mathsf{R}_{\mathsf{out}}$*.*

7. *The challenger outputs* $\mathsf{R}_{\mathsf{out}}, A, a_1, a_2$*.*

For any QPT adversary $\mathcal{A}$, $\left\|\mathsf{Game}^0_{\mathcal{A}}(1^\lambda) - \mathsf{Game}^0_{\mathcal{A}}(1^\lambda)\right\|_{Tr} \leq \mathsf{negl}(\lambda)$, assuming the existence of indistinguishability obfuscation and injective one-way functions.

Assuming the existence of subexponentially secure indistinguishability obfuscation and injective one-way functions, there exists a constant[14] *$c > 0$ such that that for any quantum adversary that runs in time $2^{-(d(\lambda))^c}$, $\left\|\mathsf{Game}^0_{\mathcal{A}}(1^\lambda) - \mathsf{Game}^0_{\mathcal{A}}(1^\lambda)\right\|_{Tr} \leq 2^{-(d(\lambda))^c}$.*

Proof. This follows by the same argument in the proof of [CLLZ21, Theorem 9]. □

Now we recall the following information-theoretic copy-protection security for coset states.

Lemma 3 (Monogamy-of-Entanglement for Coset States ([CLLZ21], implicit))**.** *Consider the following game between a tuple of adversaries $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1, \mathcal{A}_2)$ and a challenger.*

$\underline{\mathsf{Game}_{\mathcal{A}}(1^\lambda)}$

1. *The challenger samples a $\mathbb{F}_2^{d(\lambda)}$-coset as follows: Sample a subspace $A \leq \mathbb{F}_2^{d(\lambda)}$ and two elements $a_1, a_2 \in \mathbb{F}_2^{d(\lambda)}$ uniformly at random.*
2. *The challenger initialize the register R with the coset state $|A_{a_1,a_2}\rangle = \sum_{v\in A}(-1)^{\langle v,a_2\rangle}|v + a_1\rangle$.*
3. *The challenger samples a superspace B_1 of A of dimension $\frac{3d}{4}$, and a subspace B_2 of A of dimension $\frac{d}{4}$ and elements $z_1 \leftarrow B_1$, $z_2 \leftarrow B_2^\perp$. The challenger also sets $t = z_1 + a_1$ and $t' = z_2 + a_2$.*
4. *The challenger sends $\mathsf{R}, B_1, B_2, t, t'$ to the adversary $\mathcal{A}_0$.*
5. *The adversary $\mathcal{A}_0$ outputs two registers $\mathsf{R}_1, \mathsf{R}_2$.*
6. *The adversaries*[15] *$\mathcal{A}_1, \mathcal{A}_2$ receive (A, R_1) and (A, R_2), respectively.*
7. *The adversaries $\mathcal{A}_1, \mathcal{A}_2$ outputs v and w, respectively.*
8. *The challenger checks if $v = \mathsf{Can}(A, a_1)$ and $w = \mathsf{Can}(A^\perp, a_2)$. If so, it outputs 1. Otherwise, it outputs 0.*

For any (unbounded) adversary $\mathcal{A}$,

$$\Pr\left[\mathsf{Game}_{\mathcal{A}}(1^\lambda) = 1\right] \leq 2^{-c_{MoE}\cdot d(\lambda)}$$

Now we state the following information-theoretic secure leasing security for coset states.

Lemma 4 (Secure Leasing for Coset States)**.** *Consider the following game between an adversary $\mathcal{A}$ and a challenger.*

[14] Note that this is a fixed constant that cannot be improved by assuming higher levels of security for iO or OWF.
[15] The adversaries not communicating, but they are entangled through $\mathsf{R}_1, \mathsf{R}_2$

$\underline{\mathsf{Game}_{\mathcal{A}}(1^\lambda)}$

1. *The challenger samples a $\mathbb{F}_2^{d(\lambda)}$-coset as follows: Sample a subspace $A \leq \mathbb{F}_2^{d(\lambda)}$ and two elements $a_1, a_2 \in \mathbb{F}_2^{d(\lambda)}$ uniformly at random.*
2. *The challenger initialize the register R with the coset state $|A_{a_1,a_2}\rangle = \sum_{v\in A}(-1)^{\langle v,a_2\rangle}|v+a_1\rangle$.*
3. *The challenger samples a superspace B_1 of A of dimension $\frac{3d}{4}$, and a subspace B_2 of A of dimension $\frac{d}{4}$ and elements $z_1 \leftarrow B_1$, $z_2 \leftarrow B_2^\perp$. The challenger also sets $t = z_1 + a_1$ and $t' = z_2 + a_2$.*
4. *The challenger sends $\mathsf{R}, B_1, B_2, t, t'$ to the adversary $\mathcal{A}$.*
5. *The adversary $\mathcal{A}$ outputs a vector v.*
6. *The challenger submits A to the adversary $\mathcal{A}$.*
7. *The adversary $\mathcal{A}$ outputs a vector w.*
8. *The challenger checks if $v \in A + a_1$ and $w = \mathsf{Can}(A^\perp, a_2)$. If so, it outputs 1. Otherwise, it outputs 0.*

For any (unbounded) adversary $\mathcal{A}$,

$$\Pr\left[\mathsf{Game}_{\mathcal{A}}(1^\lambda) = 1\right] \leq 2^{-c_{MoE}\cdot d(\lambda)}$$

Proof. This follows directly by Lemma 3. □

Now we state the following information-theoretic LOCC leakage-resilience security for coset states.

Lemma 5 (LOCC Leakage-Resilience for Coset States ([ÇGLZR24], implicit))**.** *Consider the following parametrized game (for $i \in \{3, 4, \}$) between an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{A}_4)$ and a challenger.*

$\underline{\mathsf{Game}^i_{\mathcal{A}}(1^\lambda)}$

1. *The challenger samples a $\mathbb{F}_2^{d(\lambda)}$-coset as follows: Sample a subspace $A \leq \mathbb{F}_2^{d(\lambda)}$ and two elements $a_1, a_2 \in \mathbb{F}_2^{d(\lambda)}$ uniformly at random.*
2. *The challenger initialize the register R with the coset state $|A_{a_1,a_2}\rangle = \sum_{v\in A}(-1)^{\langle v,a_2\rangle}|v+a_1\rangle$.*
3. *The challenger samples a superspace B_1 of A of dimension $\frac{3d}{4}$, and a subspace B_2 of A of dimension $\frac{d}{4}$ and elements $z_1 \leftarrow B_1$, $z_2 \leftarrow B_2^\perp$. The challenger also sets $t = z_1 + a_1$ and $t' = z_2 + a_2$.*
4. *The challenger sends $\mathsf{R}, B_1, B_2, t, t'$ to the adversary $\mathcal{A}_1$.*
5. *The adversaries $\mathcal{A}_1, \mathcal{A}_2$ execute their LOCC protocol.*
6. *$\mathcal{A}_2$ outputs a register $\mathsf{R}_{\mathsf{out}}$.*
7. *The challenger submits $\mathsf{R}_{\mathsf{out}}, A$ to the adversary $\mathcal{A}_i$.*

8. *The adversary $\mathcal{A}_i$ outputs a vector v.*

9. *The challenger checks if $v = \mathsf{Can}(A, a_1)$ if $i = 3$ and checks if $w = \mathsf{Can}(A^{\perp}, a_2)$ if $i = 4$. If so, it outputs 1. Otherwise, it outputs 0.*

For any (unbounded) adversary $\mathcal{A}$, at least one of the following is true:

- $\Pr\left[\mathsf{Game}^3_{\mathcal{A}}(1^\lambda)\right] \leq 2^{-\frac{c_{MoE}}{2}\cdot d(\lambda)}$
- $\Pr\left[\mathsf{Game}^4_{\mathcal{A}}(1^\lambda)\right] \leq 2^{-\frac{c_{MoE}}{2}\cdot d(\lambda)}$

4.2 Quantum-Proof Reconstructive Extractors

We recall the notion of *quantum-proof* extractors, which is a randomness extractor that is secure against quantum side information on the source.

Definition 10 (Quantum-proof seeded extractor [DPVR12])**.** *A function $\mathsf{Ext} : \{0,1\}^n \times \{0,1\}^d \to \{0,1\}^m$ is said to be a (k, ϵ)-*strong quantum-proof seeded extractor *if for any cq-state $\rho \in \{0,1\}^n \otimes \mathcal{Y}$ of the registers X, Y with $H_{\mathsf{min}}(X|Y) \geq k$, we have*

$$\mathsf{Ext}(X, S), Y, S \approx_\epsilon U_m, Y, S$$

where $S \leftarrow \{0,1\}^d$ and $U_m \leftarrow \{0,1\}^m$.

We call an extractor *reconstructive extractor* if given a distinguisher between $\mathsf{Ext}(X, S), Y, S$ versus U_m, Y, S with advantage better than ϵ, there exists an (unbounded) algorithm $\mathcal{E}$ such that $\Pr[\mathcal{E}(Y) = X] \geq 2^{-k}$.

4.3 Projective and Threshold Implementations

We recall the following properties of projective and threshold implementations ([Zha20, ALL+21]).

Theorem 7 ([ALL+21])**.** *Consider the approximate threshold implementation $\mathsf{ATI}^{\epsilon,\delta}_{(\mathcal{P},\mathcal{D}),\eta}$, associated with a collection of binary projective measurements $\mathcal{P}$, a distribution $\mathcal{D}$ over the index set of $\mathcal{P}$ and a threshold value $\eta \in [0,1]$, applied to a state ρ.*

We then have the following.

- *For any state ρ,*
$$\mathrm{Tr}\left[\mathsf{ATI}^{\epsilon,\delta}_{(\mathcal{P},\mathcal{D}),\eta-\epsilon} \cdot \rho = 1\right] \geq \mathrm{Tr}\left[\mathsf{TI}_{(\mathcal{P},\mathsf{D}),\eta}\rho = 1\right] - \delta.$$
- *For any state ρ,*
$$\mathrm{Tr}\left[\mathsf{TI}_{(\mathcal{P},\mathsf{D}),\eta-\epsilon}\rho = 1\right] \geq \mathrm{Tr}\left[\mathsf{ATI}^{\epsilon,\delta}_{(\mathcal{P},\mathcal{D}),\eta}\rho = 1\right] - \delta.$$
- $\mathsf{ATI}^{\epsilon,\delta}_{(\mathcal{P},\mathcal{D}),\eta-\epsilon}$ *is efficient.*
- $\mathsf{TI}_{(\mathcal{P},\mathsf{D}),\eta}$ *is a projection and the collapsed state conditioned on outcome 1 is a mixture of eigenvectors of $\mathcal{P}$ with eigenvalue $\geq \eta$.*

We give some further generalizations below.

Theorem 8 ([ÇG24a]). *For any $k \in \mathbb{N}$, let $\mathcal{P}_\ell, \mathcal{D}_\ell$ be a collection of projective measurements and a distribution on the index set of this collection, respectively, and $\eta_\ell \in [0,1]$ be threshold values for all $\ell \in [k]$. Write $\mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta_\ell}\right)\cdot\rho\right]$ to denote the probability that the outcome of the joint measurement $\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta_\ell}$ applied on ρ is all 1, and similarly for TI.*

Then, we have the following.

- *For any k-partite state ρ,*

$$\mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta_\ell-\epsilon}\right)\rho\right] \geq \mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{TI}_{\eta_\ell}(\mathcal{P}_{\ell\mathcal{D}_\ell})\right)\rho\right] - k\cdot\delta.$$

- *For any k-partite state ρ, let ρ' be the collapsed state obtained after applying $\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta_\ell}$ to ρ and obtaining the outcome 1. Then,*

$$\mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{TI}_{\eta_\ell-2\epsilon}(\mathcal{P}_{\ell\mathcal{D}_\ell})\right)\rho'\right] \geq 1 - 2k\cdot\delta.$$

- *For any k-partite state ρ, let ρ' be the collapsed state obtained after applying $\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta}$ to ρ and obtaining the outcome 1. Then,*

$$\mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta_\ell-3\epsilon}\right)\cdot\rho'\right] \geq 1 - 3k\cdot\delta.$$

- *For any k-partite state ρ,*

$$\mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{TI}_{\eta_\ell-\epsilon}(\mathcal{P}_{\ell\mathcal{D}_\ell})\right)\rho\right] \geq \mathrm{Tr}\left[\left(\bigotimes_{\ell\in[k]} \mathsf{ATI}^{\epsilon,\delta}_{\mathcal{P}_\ell,\mathcal{D}_\ell,\eta_\ell}\right)\rho\right] - k\cdot\delta.$$

The following is implicitly shown in [ÇG24a], by adapting a proof of [Zha20].

Lemma 6 (Simulated ATI). *There exists a polynomial $q(\cdot)$ and a (parametrized) measurement $\mathsf{SimATI}^{\epsilon,\delta,\alpha}_{(P,|\mathcal{D}|),\eta}$ that receives as input a list of ℓ strings (from the support of $\mathcal{D}$) and runs in time $\mathsf{poly}(\ell)$ such that for any $0 < \epsilon, \delta, \eta, \alpha < 1$, for any collection of binary projective measurements $P = \{P_i, I - P_I\}_{i\in\mathcal{I}}$, for any possibly correlated distribution of states ρ with distributions $\mathcal{D}$ over $\mathcal{I}$,*

$$\Pr\left[\mathsf{SimATI}^{\epsilon,\delta,\alpha}_{(P,|\mathcal{D}|),\eta-5\epsilon}(s_1,\ldots,s_\ell)(\rho) = 1 : s_1,\ldots,s_\ell \leftarrow \mathcal{D}\right] \geq \Pr\left[\mathsf{ATI}^{\epsilon,\delta}_{(P,\mathcal{D}),\eta}(\rho) = 1\right] - \alpha - 4\delta.$$

$$\Pr\left[\mathsf{ATI}^{\epsilon,\delta}_{(P,\mathcal{D}),\eta-5\epsilon}(\rho) = 1\right] \geq \Pr\left[\mathsf{SimATI}^{\epsilon,\delta,\alpha}_{(P,|\mathcal{D}|),\eta}(s_1,\ldots,s_\ell)(\rho) = 1 : s_1,\ldots,s_\ell \leftarrow \mathcal{D}\right] - \alpha - 4\delta$$

$$\Pr\left[\mathsf{SimATI}^{\epsilon,\delta,\alpha}_{(P,|\mathcal{D}|),\eta-5\epsilon}(s_1,\ldots,s_\ell)(\rho) = 1 : s_1,\ldots,s_\ell \leftarrow \mathcal{D}\right] \geq \Pr\left[\mathsf{TI}_{(P,\mathcal{D}),\eta}(\rho) = 1\right] - \alpha - 4\delta$$

$$\Pr\left[\mathsf{TI}_{(P,\mathcal{D}),\eta-5\epsilon}(\rho) = 1\right] \geq \Pr\left[\mathsf{SimATI}^{\epsilon,\delta,\alpha}_{(P,|\mathcal{D}|),\eta}(s_1,\ldots,s_\ell)(\rho) = 1 : s_1,\ldots,s_\ell \leftarrow \mathcal{D}\right] - \alpha - 4\delta$$

where $\ell = q(1/\epsilon, \log(1/\delta), 1/\alpha)$. Note that SimATI depends on the parameters of $\mathcal{D}$ (e.g. length of the random tape), but otherwise does not use oracles access to $\mathcal{D}$ nor does it use any samples from $\mathcal{D}$ (except for $s_1,\ldots,s_\ell$ which are given explicitly).

5 Quantum Protection Construction for Puncturable Primitives

In this section, we give our quantum protection construction $\mathsf{QuantumProtect}$. Our construction is a generic construction does not depend on the particular primitives that is being protected, except for the size of the challenge strings and the size of the punctured keys of the primitive. Note that $\mathsf{GenState}$ does not depend even on these size parameters.

5.1 Primitives and Parameters

We give our construction, $\mathsf{QuantumProtect}$.

Let $\mathcal{F} = (\Phi, \mathsf{Eval}, \mathsf{Chal}, \mathcal{D}, \mathsf{Ver}, \mathsf{Punc})$ be the puncturable functionality to be protected. Let PRF be a pseduorandom function family scheme. All the obfuscated programs are implicitly padded to the appropriate sizes as needed in the proof.

$\underline{\mathsf{QuantumProtect.GenState}(1^\lambda)}$

1. Sample a $\mathbb{F}_2^{d(\lambda)}$-coset as follows: Sample a subspace $A \leq \mathbb{F}_2^{d(\lambda)}$ and two elements $a_1, a_2 \in \mathbb{F}_2^{d(\lambda)}$ uniformly at random.
2. Initialize the register R with the coset state $|A_{a_1,a_2}\rangle = \sum_{v\in A}(-1)^{\langle v,a_2\rangle}|v + a_1\rangle$.
3. Sample $\hat{M} \leftarrow i\mathcal{O}(M)$ where M is the membership checking program for the cosets as defined below.

   $\underline{M(b, v)}$

   Hardcoded: A, a_1, a_2

   1. If $b = 0$: Check if $v \in A + a_1$. If so, output TRUE, otherwise output FALSE, and terminate.
   2. If $b = 1$: Check if $v \in A^\perp + a_2$. If so, output TRUE, otherwise output FALSE.

4. Output $(pp = \hat{M}, \mathsf{R})$.

$\underline{\mathsf{QuantumProtect.Protect}(pp, C)}$

1. Parse $\hat{M} = pp$.
2. Sample a PRF key as $K \leftarrow \mathsf{PRF.KeyGen}(1^\lambda)$.
3. Sample $\hat{P} \leftarrow i\mathcal{O}(P)$ where P is the following program.

   $\underline{P(x, b, v)}$

   Hardcoded: $K, \hat{M}, C$

   1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\perp$ and terminate.
   2. If $b = 0$, output $C(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   3. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$.

4. Output $\hat{P}$.

QuantumProtect.Eval(R, x)

1. Parse $(\hat{P}, \mathsf{R}') = \mathsf{R}$.
2. Run $\hat{P}$ on $x, 0, \mathsf{R}'$ coherently, let y_0 denote the outcome. Then, rewind the register R' (as in Lemma 1).
3. Run $\hat{P}$ on $x, 1, \mathsf{R}'$ coherently, let y_1 denote the outcome. Then, rewind the register R' (as in Lemma 1).
4. Output $y_0 \oplus y_1$.

QuantumProtect.Delete(R) // This algorithm is only needed for secure leasing

1. Parse $(\hat{P}, \mathsf{R}') = \mathsf{R}$.
2. Measure R' in the computational basis, let *cert* denote the outcome.
3. Output *cert*.

QuantumProtect.CertVerify$(pp, cert)$ // This algorithm is only needed for secure leasing

1. Parse $\hat{M} = pp$.
2. Output $\hat{M}(0, cert)$.

Theorem 9. *For any cryptographically puncturable functionality,* QuantumProtect *satisfies strong anti-piracy (Definition 7) for all inverse polynomial $\gamma(\lambda)$.*

Theorem 10. *For any cryptographically puncturable functionality,* QuantumProtect *satisfies secure leasing security (Definition 8)*

Theorem 11. *For any cryptographically puncturable functionality,* QuantumProtect *satisfies LOCC leakage-resilience (Definition 9).*

Corollary 1. *Assuming subexponentially secure indistinguishability obfuscation and one-way functions,* QuantumProtect *is a secure copy-protection, LOCC leakage-resilience, publicly-verifiable classical-certificate secure leasing and intrusion-detection scheme for any cryptographically puncturable functionality.*

Proof. The extractor Ext used in the proof can be instantiated without any assumptions[16]. All of the other listed primitives[17] can be instantiated using subexponentially secure indistinguishability obfuscation and one-way functions for any subexponential function[18]. Correctness of the scheme is straightforward. The first three security notions follow from the theorems above, and intrusion-detection follows from secure leasing security as shown by [ÇGLZR24]. □

[16] There are many options, see e.g. [DPVR12, KK10, RRV99]
[17] See Theorem 4 for ACE with pseudorandom ciphertexts.
[18] To achieve the desired security level, we can simply instantiate the primitive with a larger security parameter.

6 Proof of Secure Leasing Security

In this section, we prove Theorem 10. We prove security through a sequence of hybrids, each of which is constructed by modifying the previous one. Throughout the hybrids, the challenger executes the codes of the algorithms of $\mathsf{QuantumProtect}$ directly (so that we can modify the execution of these algorithms).

$\underline{\mathsf{Hyb}_0}$: The original game $\mathsf{SecureLeasingGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)$.

$\underline{\mathsf{Hyb}_1}$: First, the challenger samples an ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$, $ek = \mathsf{ACE.GenEK}(sk, \mathsf{FALSE}$[19]$)$. Then it samples $r_1^*, r_2^* \leftarrow \{0,1\}^{p_4(\lambda)}$ and computes $ct_1^* = C^* \oplus G(r_1^*), ct_2^* = K \oplus G(r_2^*)$ and $a_1^{Can} = \mathsf{Can}(A, a_1), a_2^{Can} = \mathsf{Can}(A^\perp, a_2)$. Then, we define the following distribution.

$\underline{\mathcal{D}_2^{(1)}}$

Hardcoded: $cp^*, C^*, A, a_2^{Can}, ek, r_2^*$

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Set $pl_2 = 1||A||se||(\mathsf{Ext}(se, a_2^{Can}) \oplus r_2^*)||0^{\ell_4(\lambda)}$
3. Set $x^* = \mathsf{ACE.Enc}(ek, pl_2)$.
4. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
5. Output $ch^*, (x^*, sp^*)$.

Finally, we modify the way the challenger samples the challenge: It now samples it as $ch^*, x^*, sp^* \leftarrow \mathcal{D}_2^{(1)}$ instead of $ch^*, x^*, sp^* \leftarrow \mathcal{D}(cp^*, C^*)$.

$\underline{\mathsf{Hyb}_2}$: Let $Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}}(m)$ denote the following circuit.

$\underline{Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}}(m)}$

1. Parse $t||A'||se'||z||ind = m$.
2. Check if $A' = A$. If not, output TRUE and terminate.
3. If $t = 0$,
    1. Compute $r' = \mathsf{Ext}(se', a_1^{Can}) \oplus z$.
    2. Compute $C' = ct_1^* \oplus G(r')$.
    3. Check if $C = C^*$. If so, output FALSE, otherwise output TRUE and terminate.
4. If $t = 1$,
    1. Compute $r' = \mathsf{Ext}(se', a_2^{Can}) \oplus z$.
    2. Compute $K' = ct_2^* \oplus G(r')$.

[19] This represents the circuit that always outputs FALSE, meaning that the encapsulation key will not be punctured at all.

3. Check if $K' = K$. If so, output FALSE, otherwise output TRUE and terminate.

First, the challenger computes $dk' = \mathsf{ACE.GenDK}(sk, Q_{pl_1,pl_2,C^*,K,A,ct_1,ct_2,a_1^{Can},a_2^{Can}})$. We modify the way the challenger samples $\hat{P}$: Now it samples it as $\hat{P} \leftarrow i\mathcal{O}(P^{(1)})$.

$\underline{P^{(1)}(x,b,v)}$

Hardcoded: $K, \hat{M}, C^*, dk', ct_1^*, ct_2^*$

1. Check if $\hat{M}(b,v) = \mathsf{TRUE}$. If not, output $\bot$ and terminate.
2. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \bot$, parse $t||A'||se'||z||ind = pl$.
3. If $pl = \bot$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
   1. If $b = 0$, output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$ and terminate.
4. If $pl \neq \bot$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A', v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}((A')^{\bot}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $K' = ct_2^* \oplus G(r')$.
      3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

$\underline{\mathsf{Hyb}_3}$**:** Instead of setting $dk' = \mathsf{ACE.GenDK}(sk, Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}})$, the challenger now sets $dk' = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$.

$\underline{\mathsf{Hyb}_4}$**:** We modify the way the challenger verifies the deletion certificate: Instead of checking if $\hat{M}(0, cert) = \mathsf{TRUE}$, instead it directly checks if $cert \in A + a_1$.

$\underline{\mathsf{Hyb}_5}$**:** We modify the sampling of $\hat{M}$ during the execution of $\mathsf{QuantumProtect.GenState}$. We first sample a superspace B_1 of A of dimension $\frac{3d}{4}$, a subspace B_2 of A of dimension $\frac{d}{4}$ and elements $u_1 \leftarrow B_1$, $u_2 \leftarrow B_2^{\bot}$. Finally, we set $t = u_1 + a_1$ and $t' = u_2 + a_2$. Now, we sample $\hat{M} \leftarrow i\mathcal{O}(M')$ where M' is the following program.

$\underline{M'(b,v)}$

Hardcoded: t, t', B_1, B_2

1. If $b = 0$: Check if $v \in B_1 + t$. If so, output TRUE, otherwise output FALSE, and terminate.
2. If $b = 1$: Check if $v \in B_2^{\bot} + t'$. If so, output TRUE, otherwise output FALSE.

Hyb_6**:** We define the following distribution.

$\mathcal{D}_2^{(2)}$

Hardcoded: cp^*, C^*, A, ek

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Sample $r^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$.
3. Set $pl_2 = 1||A||se||r^{**}||0^{\ell_4(\lambda)}$.
4. Set $x^* = \mathsf{ACE.Enc}(ek, pl_2)$.
5. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
6. Output $ch^*, (x^*, sp^*)$.

Finally, we modify the way the challenger samples the challenge: It now samples it as $ch^*, x^*, sp^* \leftarrow \mathcal{D}_2^{(2)}$ instead of $ch^*, x^*, sp^* \leftarrow \mathcal{D}_2^{(1)}(cp^*, C^*)$. We will also write $act^* = x^*$ since x^* is an ACE ciphertext.

Hyb_7**:** We modify[20] the sampling of $\hat{M}$ during the execution of $\mathsf{QuantumProtect.GenState}$: It is now sampled as $\hat{M} \leftarrow i\mathcal{O}(M)$.

Hyb_8**:** We define this hybrid by reordering some independent steps of Hyb_7.

Hyb_8

1. Chal and $\mathcal{A}$ interact, Chal outputs C^* and cp^*.
2. The challenger samples $(\hat{M}, \mathsf{R}') \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$. Let A, a_1, a_2 denote the coset sampled during this sampling.
3. The challenger samples $K \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.
4. The challenger samples $r_1^*, r_2^* \leftarrow \{0,1\}^{p_4(\lambda)}$ and computes $ct_1^* = C^* \oplus G(r_1^*), ct_2^* = K \oplus G(r_2^*)$.
5. The challenger samples ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$ and $dk' = \mathsf{ACE.GenEK}(sk, \mathsf{FALSE})$.
6. The challenger samples $\hat{P} \leftarrow i\mathcal{O}(P^{(1)})$.
7. $\mathcal{A}$ receives $\hat{P}, \mathsf{R}'$.
8. $\mathcal{A}$ outputs a certificate $cert$.
9. The challenger samples $ch^*, (act^*, sp^*) \leftarrow \mathcal{D}_2^{(2)}(cp^*, C^*)$.
10. $\mathcal{A}$ receives ch^* and outputs y^*.

[20] Essentially we are undoing the change in Hyb_5

11. The challenger outputs 1 if $cert \in A + a_1$ and $\mathsf{Ver}(sp^*, y^*) = \mathsf{TRUE}$; otherwise, it outputs 0.

Lemma 7. $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_8$.

We prove this lemma in Section 6.1.

Lemma 8. $\Pr[\mathsf{Hyb}_8 = 1] \leq \mathsf{negl}(\lambda)$.

Proof. This follows directly by Theorem 12. Here, our adversary for Theorem 12 always outputs $c' = 2$. □

Finally, we complete the proof of Theorem 10. Suppose for a contradiction that there exists a QPT adversary $\mathcal{A}$ and a polynomial $q(\cdot)$ such that $\Pr\left[\mathsf{SecureLeasingGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \geq \frac{1}{q(\lambda)}$ for an infinite number of values $\lambda > 0$. Then, by Lemma 7, we get $\Pr[\mathsf{Hyb}_7 = 1] \geq \frac{1}{2q(\lambda)}$ for an infinite number of values $\lambda > 0$. However, this is a contradiction to Lemma 8.

6.1 Indistinguishability of Hybrids: Proof of Lemma 7

Lemma 9. $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_1$.

Proof. Observe that the experiments do not use the decryption key dk of ACE. Thus, the result follows by the pseudorandom ciphertext property of ACE and the pseudorandom puncturing point property of $\mathcal{F}$. □

Lemma 10. $\mathsf{Hyb}_1 \approx \mathsf{Hyb}_2$.

Proof. We claim that the programs P and $P^{(1)}$ have the exact same functionality. Observe that once we prove this, the result follows by security of $i\mathcal{O}$.

Now we prove our claim. Suppose for a contradiction that there is some input x, b, v such that $P(x, b, v) \neq P^{(1)}(x, b, v)$. First, it is easy to see that if $\mathsf{ACE.Dec}(dk', x) = \perp$, then we have $P(x, b, v) = P^{(1)}(x, b, v)$. Thus, we assume $\mathsf{ACE.Dec}(dk', x) \neq \perp$. However, then we have that pl satisfies $Q(pl) = \mathsf{FALSE}$ during the execution of $P^{(1)}(x, b, v)$, by the constrained decryption safety of ACE. It is also easy to see that $P(x, b, v) = P^{(1)}(x, b, v)$ if $b = 0 \wedge t = 1$ or $b = 1 \wedge t = 0$ where t is the first bit of $\mathsf{ACE.Dec}(dk', x)$. Thus, at this point, we know that during the execution of $P^{(1)}(x, b, v)$, we have

- $Q(pl) = \mathsf{FALSE}$ and
- $(b, t) = (0, 0) \vee (b, t) = (1, 1)$,

We will consider the two cases separately. However, first we establish a fact about pl, given that $Q(pl) = \mathsf{FALSE}$: Let $t||A'||se'||z$ be the parsing of pl that $P^{(1)}(x, b, v)$ obtains during its execution - then, we have $A' = A$.

Now assume that $(b, t) = (0, 0)$. Then, $P^{(1)}(x, b, v)$ enters the first subroutine (i.e. $t = 0$ case) after verifying $pl \neq \perp$. Then, we get $a' = a_1^{Can}$ when the circuit computes $a' = \mathsf{Can}(A', v)$ since $A' = A$ and $v \in A + a_1$ (since $\hat{M}(0, v) = \mathsf{TRUE}$ and $i\mathcal{O}$ satisfies correctness). Then, we get $C' = C^*$ since $Q(pl) = \mathsf{FALSE}$. Thus, we get that $P^{(1)}(x, b, v) = C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$. However, we also have $P(x, b, v) = C(x) \oplus \mathsf{PRF.Eval}(K, x)$, which is a contradiction to our assumption that $P(x, b, v) \neq P^{(1)}(x, b, v)$.

Finally, now assume $(b, t) = (1, 1)$. Then, $P^{(1)}(x, b, v)$ enters the second subroutine (i.e. $t = 1$ case) after verifying $pl \neq \perp$. Then, we get $a' = a_2^{Can}$ when the circuit computes $a' = \mathsf{Can}((A')^\perp, v)$

since $A' = A$ and $v \in A^\perp + a_2$ (since $\hat{M}(1, v) = \mathsf{TRUE}$ and $i\mathcal{O}$ satisfies correctness). Then, we get $K' = K$ since $Q(pl) = \mathsf{FALSE}$. Finally, we get that $P^{(1)}(x, b, v) = \mathsf{PRF.Eval}(K, x)$. However, we also have $P(x, b, v) = \mathsf{PRF.Eval}(K, x)$, which is a contradiction to our assumption that $P(x, b, v) \neq P^{(1)}(x, b, v)$.

Thus, we conclude that $P(x, b, v) = P^{(1)}(x, b, v)$ for all x, b, v. Then the result follows by security of $i\mathcal{O}$. □

Lemma 11. $\mathsf{Hyb}_2 \approx \mathsf{Hyb}_3$.

Proof. Observe that these experiments only use the ACE ciphertext $\mathsf{ACE.Enc}(ek, pl_2)$, and do not use the encapsulation key ek anywhere else. Thus, by security of constrained decapsulation keys of ACE, the keys $\mathsf{ACE.GenDK}(sk, Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}}))$ and $\mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$ are indistinguishable since $Q(pl_2) = \mathsf{FALSE}$. Hence, the result follows. □

Lemma 12. $\mathsf{Hyb}_3 \approx \mathsf{Hyb}_4$.

Proof. This follows by the correctness of $i\mathcal{O}$. □

Lemma 13. $\mathsf{Hyb}_4 \approx \mathsf{Hyb}_5$.

Proof. Follows directly by Lemma 2. □

Lemma 14. $\mathsf{Hyb}_5 \approx \mathsf{Hyb}_6$.

Proof. Suppose for a contradiction that $\mathsf{Hyb}_5 \not\approx \mathsf{Hyb}_6$. Then, this gives a distinguisher between $se, \mathsf{Ext}(se, a_2^{Can})$ and se, r^{**} where $r^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$, $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$, using the internal state of $\mathcal{A}$ (right before it receives the challenge ch^*, x^*, sp^*) and cp^*, C^*, ek and A. Then, by reconstructive extractor security, there exists an (unbounded) predictor algorithm $\mathcal{E}$ that output a_2^{Can} with probability $> 2^{-\frac{c_{MoE}}{4}\lambda}$ given the internal state of $\mathcal{A}$ and cp^*, C^*, ek and A.

This gives us the following adversary $\mathcal{B}$ for the secure leasing game for coset states (Lemma 4).

$\underline{\mathcal{B}(1^\lambda)}$

1. **Setup Phase:** Receive $\mathsf{R}, B_1, B_2, t, t'$ from the challenger.
2. Simulate both the challenger and the adversary of Hyb_5 (or Hyb_6, since they are equivalent up to that point) until $\mathcal{A}$ outputs the deletion certificate $cert$. Let $\mathsf{R}_{\mathsf{int}}$ denote the internal state of $\mathcal{A}$ at this point.
3. **Deletion Phase:** Output $v = cert$.
4. **Challenge Phase:** Receive A from the challenger. Run $\mathcal{E}(\mathsf{R}_{\mathsf{int}}, ch^*, x^*, sp^*, A)$ to obtain w. Output w.

Crucially note that simulating both the challenger and the adversary of Hyb_5 until $\mathcal{A}$ outputs the deletion certificate $cert$ does not need the subspace description A. Thus, $\mathcal{B}$ is a valid adversary for the secure leasing game for coset states

By the guarantee for the predictor algorithm $\mathcal{E}$ that was discussed above, we get that $\mathcal{B}$ wins the secure leasing game for coset states with probability $> 2^{-\frac{c_{MoE}}{4}\lambda}$, which is a contradiction by (Lemma 4). □

Lemma 15. $\mathsf{Hyb}_6 \approx \mathsf{Hyb}_7$.

Proof. Follows directly by Lemma 2. □

Lemma 16. $\mathsf{Hyb}_7 \approx \mathsf{Hyb}_8$.

Proof. Reordering independent steps makes no difference. □

7 Proof of LOCC Leakage-Resilience Security

In this section, we prove Theorem 11. The proof is highly similar to the proof of Theorem 10, with the major difference being Lemma 23 which relies on the LOCC leakage-resilience property for coset states (Lemma 5) whereas the analogus part of the proof of Theorem 10 relies on the secure leasing property of coset states (Lemma 4).

We prove security through a sequence of hybrids, each of which is constructed by modifying the previous one. Throughout the hybrids, the challenger executes the codes of the algorithms of $\mathsf{QuantumProtect}$ directly (so that we can modify the execution of these algorithms).

We define the following hybrids, which are parametrized by $c' \in \{1,2\}$.

$\underline{\mathsf{Hyb}_0}$**:** The original game $\mathsf{LeakageGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)$.

$\underline{\mathsf{Hyb}_{1,c'}}$**:** First, the challenger samples an ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$, $ek = \mathsf{ACE.GenEK}(sk, \mathsf{FALSE}$[21]$)$. Then it samples $r_1^*, r_2^* \leftarrow \{0,1\}^{p_4(\lambda)}$ and computes $ct_1^* = C^* \oplus G(r_1^*), ct_2^* = K \oplus G(r_2^*)$ and $a_1^{Can} = \mathsf{Can}(A, a_1), a_2^{Can} = \mathsf{Can}(A^\perp, a_2)$. Then, we define the following distributions.

$\underline{\mathcal{D}_1^{(1)}}$

Hardcoded: $cp^*, C^*, A, a_1^{Can}, ek, r_1^*$

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Set $pl_1 = 0||A||se||(\mathsf{Ext}(se, a_1^{Can}) \oplus r_1^*)||0^{\ell_4(\lambda)}$
3. Set $x^* = \mathsf{ACE.Enc}(ek, pl_1)$.
4. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
5. Output $ch^*, (x^*, sp^*)$.

$\underline{\mathcal{D}_2^{(1)}}$

Hardcoded: $cp^*, C^*, A, a_2^{Can}, ek, r_2^*$

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Set $pl_2 = 1||A||se||(\mathsf{Ext}(se, a_2^{Can}) \oplus r_2^*)||0^{\ell_4(\lambda)}$
3. Set $x^* = \mathsf{ACE.Enc}(ek, pl_2)$.
4. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.

[21] This represents the circuit that always outputs FALSE, meaning that the encapsulation key will not be punctured at all.

5. Output $ch^*, (x^*, sp^*)$.

Finally, we modify the way the challenger samples the challenge: It now samples it as $ch^*, x^*, sp^* \leftarrow \mathcal{D}^{(1)}_{c'}$ instead of $ch^*, x^*, sp^* \leftarrow \mathcal{D}(cp^*, C^*)$.

$\mathsf{Hyb}_{2,c'}$**:** Let $Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}}(m)$ denote the following circuit.

$Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}}(m)$

1. Parse $t||A'||se'||z||ind = m$.
2. Check if $A' = A$. If not, output TRUE and terminate.
3. If $t = 0$,
   1. Compute $r' = \mathsf{Ext}(se', a_1^{Can}) \oplus z$.
   2. Compute $C' = ct_1^* \oplus G(r')$.
   3. Check if $C = C^*$. If so, output FALSE, otherwise output TRUE and terminate.
4. If $t = 1$,
   1. Compute $r' = \mathsf{Ext}(se', a_2^{Can}) \oplus z$.
   2. Compute $K' = ct_2^* \oplus G(r')$.
   3. Check if $K' = K$. If so, output FALSE, otherwise output TRUE and terminate.

First, the challenger computes $dk' = \mathsf{ACE.GenDK}(sk, Q_{pl_1,pl_2,C^*,K,A,ct_1,ct_2,a_1^{Can},a_2^{Can}})$. We modify the way the challenger samples $\hat{P}$: Now it samples it as $\hat{P} \leftarrow i\mathcal{O}(P^{(1)})$.

$P^{(1)}(x, b, v)$

Hardcoded: $K, \hat{M}, C^*, dk', ct_1^*, ct_2^*$

1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\perp$ and terminate.
2. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \perp$, parse $t||A'||se'||z||ind = pl$.
3. If $pl = \perp$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
   1. If $b = 0$, output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$ and terminate.
4. If $pl \neq \perp$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A', v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}((A')^{\perp}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.

2. Compute $K' = ct_2^* \oplus G(r')$.
3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

$\mathsf{Hyb}_{3,c'}$: Instead of setting $dk' = \mathsf{ACE.GenDK}(sk, Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}})$, the challenger now sets $dk' = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$.

$\mathsf{Hyb}_{4,c'}$: We modify the sampling of $\hat{M}$ during the execution of $\mathsf{QuantumProtect.GenState}$. We first sample a superspace B_1 of A of dimension $\frac{3d}{4}$, a subspace B_2 of A of dimension $\frac{d}{4}$ and elements $u_1 \leftarrow B_1$, $u_2 \leftarrow B_2^{\perp}$. Finally, we set $t = u_1 + a_1$ and $t' = u_2 + a_2$. Now, we sample $\hat{M} \leftarrow i\mathcal{O}(M')$ where M' is the following program.

$M'(b, v)$

Hardcoded: t, t', B_1, B_2

1. If $b = 0$: Check if $v \in B_1 + t$. If so, output TRUE, otherwise output FALSE, and terminate.
2. If $b = 1$: Check if $v \in B_2^{\perp} + t'$. If so, output TRUE, otherwise output FALSE.

$\mathsf{Hyb}_{5,c'}$: We define the following distributions.

$\mathcal{D}_1^{(2)}$

Hardcoded: cp^*, C^*, A, ek

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Sample $r_1^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$.
3. Set $pl_1 = 0||A||se||r_1^{**}||0^{\ell_4(\lambda)}$.
4. Set $x^* = \mathsf{ACE.Enc}(ek, pl_1)$.
5. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
6. Output $ch^*, (x^*, sp^*)$.

$\mathcal{D}_2^{(2)}$

Hardcoded: cp^*, C^*, A, ek

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Sample $r_1^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$.
3. Set $pl_2 = 1||A||se||r_2^{**}||0^{\ell_4(\lambda)}$.
4. Set $x^* = \mathsf{ACE.Enc}(ek, pl_2)$.
5. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.

6. Output $ch^*, (x^*, sp^*)$.

Finally, we modify the way the challenger samples the challenge: It now samples it as $ch^*, x^*, sp^* \leftarrow \mathcal{D}^{(2)}_{c'}$ instead of $ch^*, x^*, sp^* \leftarrow \mathcal{D}^{(1)}_{c'}(cp^*, C^*)$. We will also write $act^* = x^*$ since x^* is an ACE ciphertext.

$\mathsf{Hyb}_{6,c'}$**:** We modify[22] the sampling of $\hat{M}$ during the execution of $\mathsf{QuantumProtect.GenState}$: It is now sampled as $\hat{M} \leftarrow i\mathcal{O}(M)$.

$\mathsf{Hyb}_{7,c'}$**:** We define this hybrid by reordering some independent steps of $\mathsf{Hyb}_{6,c'}$.

$\mathsf{Hyb}_{7,c'}$

1. Chal and $\mathcal{A}$ interact, Chal outputs C^* and cp^*.
2. The challenger samples $(\hat{M}, \mathsf{R}') \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$. Let A, a_1, a_2 denote the coset sampled during this sampling.
3. The challenger samples $K \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.
4. The challenger samples $r^*_1, r^*_2 \leftarrow \{0,1\}^{p_4(\lambda)}$ and computes $ct^*_1 = C^* \oplus G(r^*_1), ct^*_2 = K \oplus G(r^*_2)$.
5. The challenger samples ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$ and $dk' = \mathsf{ACE.GenEK}(sk, \mathsf{FALSE})$.
6. The challenger samples $\hat{P} \leftarrow i\mathcal{O}(P^{(1)})$.
7. $\mathcal{A}$ receives $\hat{P}, \mathsf{R}'$.
8. $\mathcal{A}$ outputs a certificate $cert$.
9. The challenger samples $ch^*, (act^*, sp^*) \leftarrow \mathcal{D}^{(2)}_{c'}(cp^*, C^*)$.
10. $\mathcal{A}$ receives ch^* and outputs y^*.
11. The challenger outputs 1 if $\mathsf{Ver}(sp^*, y^*) = \mathsf{TRUE}$; otherwise, it outputs 0.

Lemma 17. *For any fixed QPT adversary $\mathcal{A}$, at least one of the following is true:*

- $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_{7,1}$.
- $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_{7,2}$.

We prove this lemma in Section 7.1.

Lemma 18. $\Pr[\mathsf{Hyb}_{7,1} = 1] \leq \mathsf{negl}(\lambda)$. $\Pr[\mathsf{Hyb}_{7,2} = 1] \leq \mathsf{negl}(\lambda)$.

Proof. This follows directly by Theorem 12, invoked with two different adversaries, where the first one always outputs $c' = 1$ and the second one always outputs $c' = 2$. □

[22] Essentially we are undoing the change in Hyb_5

Finally, we complete the proof of Theorem 11. Suppose for a contradiction that there exists a QPT adversary $\mathcal{A}$ and a polynomial $q(\cdot)$ such that $\Pr\left[\mathsf{LeakageGame}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)=1\right] \geq \frac{1}{q(\lambda)}$ for an infinite number of values $\lambda > 0$. Then, by Lemma 17, we get either $\Pr\left[\mathsf{Hyb}_{7,1}=1\right] \geq \frac{1}{2q(\lambda)}$ for an infinite number of values $\lambda > 0$, or $\Pr\left[\mathsf{Hyb}_{7,2}=1\right] \geq \frac{1}{2q(\lambda)}$ for an infinite number of values $\lambda > 0$. However, this is a contradiction to Lemma 18.

7.1 Indistinguishability of Hybrids: Proof of Lemma 17

Lemma 19. $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_{1,1}$ *and* $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_{1,2}$.

Proof. Observe that the experiments do not use the decryption key dk of ACE. Thus, the result follows by the pseudorandom ciphertext property of ACE and the pseudorandom puncturing point property of $\mathcal{F}$. □

Lemma 20. $\mathsf{Hyb}_{1,1} \approx \mathsf{Hyb}_{2,1}$ *and* $\mathsf{Hyb}_{1,2} \approx \mathsf{Hyb}_{2,2}$..

Proof. Both cases have the same proof.

We claim that the programs P and $P^{(1)}$ have the exact same functionality. Observe that once we prove this, the result follows by security of $i\mathcal{O}$.

Now we prove our claim. Suppose for a contradiction that there is some input x, b, v such that $P(x,b,v) \neq P^{(1)}(x,b,v)$. First, it is easy to see that if $\mathsf{ACE.Dec}(dk', x) = \perp$, then we have $P(x,b,v) = P^{(1)}(x,b,v)$. Thus, we assume $\mathsf{ACE.Dec}(dk', x) \neq \perp$. However, then we have that pl satisfies $Q(pl) = \mathsf{FALSE}$ during the execution of $P^{(1)}(x,b,v)$, by the constrained decryption safety of ACE. It is also easy to see that $P(x,b,v) = P^{(1)}(x,b,v)$ if $b = 0 \wedge t = 1$ or $b = 1 \wedge t = 0$ where t is the first bit of $\mathsf{ACE.Dec}(dk', x)$. Thus, at this point, we know that during the execution of $P^{(1)}(x,b,v)$, we have

- $Q(pl) = \mathsf{FALSE}$ and
- $(b,t) = (0,0) \vee (b,t) = (1,1)$,

We will consider the two cases separately. However, first we establish a fact about pl, given that $Q(pl) = \mathsf{FALSE}$: Let $t||A'||se'||z$ be the parsing of pl that $P^{(1)}(x,b,v)$ obtains during its execution - then, we have $A' = A$.

Now assume that $(b,t) = (0,0)$. Then, $P^{(1)}(x,b,v)$ enters the first subroutine (i.e. $t = 0$ case) after verifying $pl \neq \perp$. Then, we get $a' = a_1^{Can}$ when the circuit computes $a' = \mathsf{Can}(A', v)$ since $A' = A$ and $v \in A + a_1$ (since $\hat{M}(0,v) = \mathsf{TRUE}$ and $i\mathcal{O}$ satisfies correctness). Then, we get $C' = C^*$ since $Q(pl) = \mathsf{FALSE}$. Thus, we get that $P^{(1)}(x,b,v) = C^*(x) \oplus \mathsf{PRF.Eval}(K,x)$. However, we also have $P(x,b,v) = C(x) \oplus \mathsf{PRF.Eval}(K,x)$, which is a contradiction to our assumption that $P(x,b,v) \neq P^{(1)}(x,b,v)$.

Finally, now assume $(b,t) = (1,1)$. Then, $P^{(1)}(x,b,v)$ enters the second subroutine (i.e. $t = 1$ case) after verifying $pl \neq \perp$. Then, we get $a' = a_2^{Can}$ when the circuit computes $a' = \mathsf{Can}((A')^\perp, v)$ since $A' = A$ and $v \in A^\perp + a_2$ (since $\hat{M}(1,v) = \mathsf{TRUE}$ and $i\mathcal{O}$ satisfies correctness). Then, we get $K' = K$ since $Q(pl) = \mathsf{FALSE}$. Finally, we get that $P^{(1)}(x,b,v) = \mathsf{PRF.Eval}(K,x)$. However, we also have $P(x,b,v) = \mathsf{PRF.Eval}(K,x)$, which is a contradiction to our assumption that $P(x,b,v) \neq P^{(1)}(x,b,v)$.

Thus, we conclude that $P(x,b,v) = P^{(1)}(x,b,v)$ for all x, b, v. Then the result follows by security of $i\mathcal{O}$. □

Lemma 21. $\mathsf{Hyb}_{2,1} \approx \mathsf{Hyb}_{3,1}$ *and* $\mathsf{Hyb}_{2,2} \approx \mathsf{Hyb}_{3,2}$.

Proof. Let $c' \in \{1,2\}$. Observe that these experiment $\mathsf{Hyb}_{2,c'}$ only use the ACE ciphertext $\mathsf{ACE.Enc}(ek, pl_{c'})$, and does not use the encapsulation key ek anywhere else. Thus, by security of constrained decapsulation keys of ACE, the keys $\mathsf{ACE.GenDK}(sk, Q_{C^*,K,A,ct_1^*,ct_2^*,a_1^{Can},a_2^{Can}}))$ and $\mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$ are indistinguishable since $Q(pl_2) = \mathsf{FALSE}$. Hence, the result follows. □

Lemma 22. $\mathsf{Hyb}_{3,1} \approx \mathsf{Hyb}_{4,1}$ *and* $\mathsf{Hyb}_{3,2} \approx \mathsf{Hyb}_{4,2}$.

Proof. Follows directly by Lemma 2. □

Lemma 23. *For any fixed adversary $\mathcal{A}$, either* $\mathsf{Hyb}_{4,1} \approx \mathsf{Hyb}_{5,1}$ ***or*** $\mathsf{Hyb}_{4,2} \approx \mathsf{Hyb}_{5,2}$.

Proof. Suppose for a contradiction there exists an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ such that $\mathsf{Hyb}_{4,1} \not\approx \mathsf{Hyb}_{5,1}$ **or** $\mathsf{Hyb}_{4,2} \not\approx \mathsf{Hyb}_{5,2}$.

First, observe that $\mathsf{Hyb}_{4,1}$ and $\mathsf{Hyb}_{4,2}$ are exactly the same until the challenge phase. Now write $\mathsf{R}_{\mathsf{int}}$ to denote the internal state of $\mathcal{A}_2$ right before it receives the challenge (equivalently, right after the LOCC protocol between $\mathcal{A}_1$ and $\mathcal{A}_2$ concludes).

$\mathsf{Hyb}_{4,1} \not\approx \mathsf{Hyb}_{5,1}$ implies that there exists a distinguisher between $se, \mathsf{Ext}(se, a_1^{Can})$ and se, r^{**} where $r^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$, $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$, using $\mathsf{R}_{\mathsf{out}}$ and cp^*, C^*, ek and A. Then, by reconstructive extractor security of Ext, there exists an (unbounded) predictor algorithm $\mathcal{E}_1$ that outputs a_1^{Can} with probability $> 2^{-\frac{c_{MoE}}{4}\lambda}$ given $\mathsf{R}_{\mathsf{out}}$ and cp^*, C^*, ek and A.

Similarly, $\mathsf{Hyb}_{4,2} \not\approx \mathsf{Hyb}_{5,2}$ implies that there exists a distinguisher between $se, \mathsf{Ext}(se, a_2^{Can})$ and se, r^{**} where $r^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$, $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$, using $\mathsf{R}_{\mathsf{out}}$ and cp^*, C^*, ek and A. Then, by reconstructive extractor security of Ext, there exists an (unbounded) predictor algorithm $\mathcal{E}_2$ that output a_2^{Can} with probability $> 2^{-\frac{c_{MoE}}{4}\lambda}$ given $\mathsf{R}_{\mathsf{out}}$ and cp^*, C^*, ek and A.

Now consider the following adversaries $\mathcal{B}_1, \mathcal{B}_2$ for the LOCC leakage-resilience for coset states (Lemma 5).

$\underline{\mathcal{B}_1(1^\lambda)}$

1. **Setup Phase:** Receive $\mathsf{R}, B_1, B_2, t, t'$ from the challenger.
2. **Leakage Phase:** Simulate[a] both the challenger and the adversary $\mathcal{A}_1$ of $\mathsf{Hyb}_{4,1}$ (or $\mathsf{Hyb}_{4,2}, \mathsf{Hyb}_{5,1}, \mathsf{Hyb}_{5,2}$, since they are all equivalent up to that point) until the end of the LOCC protocol between $\mathcal{A}_1$ and $\mathcal{A}_2$. After that, send cp^*, C^*, ek to $\mathcal{B}_2$.

[a]Note that $\mathcal{B}_1$ is not simulating $\mathcal{A}_2$, it is directly receiving messages from $\mathcal{B}_2$ in the LOCC leakage-resilience game for coset states. However, as we see below, $\mathcal{B}_2$ will simulate $\mathcal{A}_2$.

$\underline{\mathcal{B}_2(1^\lambda)}$

1. Simulate $\mathcal{A}_2$ and outputs its internal state at the end of the LOCC protocol (i.e., right before it receives its challenge), along with cp^*, C^*, ek which is received from $\mathcal{B}_1$ as an additional round at the end.

Crucially note that simulating both the challenger and the adversary of $\mathsf{Hyb}_{4,1}$ until the LOCC protocol is done does not require the subspace description A. Thus, $\mathcal{B}_1$ is a valid adversary for the LOCC leakage-resilience game for coset states .

By the guarantees for the predictor algorithm $\mathcal{E}_1, \mathcal{E}_2$ that was discussed above, we get that $(\mathcal{B}_1, \mathcal{B}_2, \mathcal{E}_1, \mathcal{E}_2)$ constitutes a contradiction for Lemma 5. □

Lemma 24. $\mathsf{Hyb}_{5,1} \approx \mathsf{Hyb}_{6,1}$ *and* $\mathsf{Hyb}_{5,2} \approx \mathsf{Hyb}_{6,2}$.

Proof. Follows directly by Lemma 2. □

Lemma 25. $\mathsf{Hyb}_{6,1} \equiv \mathsf{Hyb}_{7,1}$ *and* $\mathsf{Hyb}_{6,2} \equiv \mathsf{Hyb}_{7,2}$.

Proof. Reordering independent steps makes no difference. □

8 Proof of Copy-Protection Security

We first describe the primitives we use in the proof. All primitives are assumed to be subexponential-advantage secure against subexponential-time adversaries (precise values will be clear from the proof). Let ACE be an asymmetrically constrainable encapsulation scheme with pseudorandom ciphertexts, let G be a pseudorandom generator. Let Ext be a constructive extractor against quantum side-information (Section 4.2) for sources that are 2^{-k} unpredictable sources. Let ϵ_{Ext} denote the extractor error level. Let $p_1(\lambda)$ denote the input size of the circuits of $\mathcal{F}$, that is, $\mathcal{X} = \{0,1\}^{p_1(\lambda)}$. Let $p_5(\lambda)$ denote the output size of the circuits of $\mathcal{F}$. Let $p_2(\lambda)$ denote the maximum of the circuit size of $\mathcal{F}$ and the key length of the PRF scheme PRF. Let $\alpha_2(\lambda)$ denote the security of ACE, let $\alpha_3(\lambda)$ denote iO security, let $\alpha_4(\lambda)$ denote the security level from Lemma 2. $\alpha_1(\lambda)$ is a parameter that will be clear from the proof

In this section, we prove Theorem 9. We prove security through a sequence of hybrids, each of which is constructed by modifying the previous one. Throughout the hybrids, the challenger executes the codes of the algorithms of $\mathsf{QuantumProtect}$ directly (so that we can modify the execution of these algorithms). Finally, we also implicitly assume that the functionality $\mathcal{F}$ has random puncturing points rather than pseudorandom puncturing points: This assumption is without loss of generality since we can simply insert an hybrid at the beginning where switch the threshold implementation for $\mathcal{D}$ to the version where the puncturing points are truly random (thanks to pseudorandom puncturing property of $\mathcal{F}$ and Lemma 6).

Suppose for a contradiction that there exists a QPT adversary $\mathcal{A}$ and polynomials $q_1(\lambda), q_2(\lambda)$ such that $\Pr\left[\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \geq \frac{1}{q_1(\lambda)}$ for infinitely many values of $\lambda > 0$, where $\gamma(\lambda) = \frac{1}{q_2(\lambda)}$. We also set the parameters $\epsilon^*(\lambda) = \frac{\gamma(\lambda)}{128}$. $\delta^*(\lambda)$ will be chosen to satisfy the parameter constraints given in the proof.

$\underline{\mathsf{Hyb}_0}$**:** The original security game $\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda)$. As a notation, we write $b^1_{Test,1}$ and $b^1_{Test,2}$ to denote the measurement outcomes obtained by the challenger by applying the threshold implementations to the registers R_1 and R_2, respectively. We set $b^1_{Test} = b^1_{Test,1} \wedge b^1_{Test,2}$. Thus, outcome of the experiment in this hybrid is b^1_{Test}.

$\underline{\mathsf{Hyb}_1}$**:** Instead of applying the threshold implementation $\mathsf{TI}_{\mathcal{D}(cp^*,C^*),p^{\mathcal{F}}_{triv}+\gamma(\lambda)}$ on the registers $\mathsf{R}_1, \mathsf{R}_2$, the challenger now instead applies approximate threshold implementation $\mathsf{ATI}^{(1)} = \mathsf{ATI}^{\epsilon^*,\delta^*}_{\mathcal{D}(cp^*,C^*),p^{\mathcal{F}}_{triv}+\frac{126\gamma}{128}}$ on the registers R_1 and R_2. We still write $b^1_{Test,1}, b^1_{Test,2}$ to denote the measurement outcomes, respectively. Note that the output of the game is b^1_{Test} in this hybrid.

$\underline{\mathsf{Hyb}_2}$**:** We modify the sampling of $\hat{M}$ during the execution of $\mathsf{QuantumProtect.GenState}$. We first sample a superspace B_1 of A of dimension $\frac{3d}{4}$, a subspace B_2 of A of dimension $\frac{d}{4}$ and elements $u_1 \leftarrow B_1$, $u_2 \leftarrow B_2^\perp$. Finally, we set $t = u_1 + a_1$ and $t' = u_2 + a_2$. Now, we sample $\hat{M} \leftarrow i\mathcal{O}(M')$ where M' is the following program.

$M'(b, v)$

Hardcoded: t, t', B_1, B_2

1. If $b = 0$: Check if $v \in B_1 + t$. If so, output TRUE, otherwise output FALSE, and terminate.
2. If $b = 1$: Check if $v \in B_2^{\perp} + t'$. If so, output TRUE, otherwise output FALSE.

Hyb_3**:** After applying $\mathsf{ATI}^{(1)} = \mathsf{ATI}^{\epsilon^*,\delta^*}_{\mathcal{D}(cp^*,C^*),p^{\mathcal{F}}_{triv}+\frac{126\gamma}{128}}$ on the registers R_1 and R_2, the challenger applies $\mathsf{TI}^{(2)} \otimes \mathsf{TI}^{(2)}$ to the registers $\mathsf{R}_1, \mathsf{R}_2$ where $\mathsf{TI}^{(2)} = \mathsf{TI}_{\mathcal{D}(cp^*,C^*),p^{\mathcal{F}}_{triv}+\frac{122\gamma}{128}}$. We write $b^2_{Test,1}, b^2_{Test,2}$ to denote the measurement outcomes, respectively. Finally, the challenger now outputs $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2}$ as the output of the game.

Hyb_4**:** The challenger samples an ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$, $ek = \mathsf{ACE.GenEK}(sk, \mathsf{FALSE}$[23]$)$ and $dk' = \mathsf{ACE.GenDK}(sk, \mathsf{TRUE}$[24]$)$. Then it samples $r^*_1, r^*_2 \leftarrow \{0,1\}^{p_4(\lambda)}$ and computes $ct^*_1 = C^* \oplus G(r^*_1), ct^*_2 = K \oplus G(r^*_2)$ and $a^{Can}_1 = \mathsf{Can}(A, a_1), a^{Can}_2 = \mathsf{Can}(A^{\perp}, a_2)$. We also modify the way the challenger samples $\hat{P}$: Now it samples it as $\hat{P} \leftarrow i\mathcal{O}(P^{(1)})$.

$P^{(1)}(x, b, v)$

Hardcoded: $K, \hat{M}, C^*, dk', ct^*_1, ct^*_2$

1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\perp$ and terminate.
2. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \perp$, parse $t||A'||se'||z||ind = pl$.
3. If $pl = \perp$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
    1. If $b = 0$, output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
    2. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$ and terminate.
4. If $pl \neq \perp$,
    1. If $t = 0$,
        1. Compute $a' = \mathsf{Can}(A', v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
        2. Compute $C' = ct^*_1 \oplus G(r')$.
        3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
    2. If $t = 1$,
        1. Compute $a' = \mathsf{Can}((A')^{\perp}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
        2. Compute $K' = ct^*_2 \oplus G(r')$.
        3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

Hyb_5**:** The challenger now sets $dk' = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$.

[23] This represents the circuit that always outputs FALSE, meaning that the encapsulation key will not be punctured at all.

[24] This represents the circuit that always outputs TRUE, meaning that the encapsulation key will be punctured everywhere.

$\underline{\mathsf{Hyb}_6}$**:** We define the following distributions where

- $ek_1 = \mathsf{ACE.GenEK}(sk, PRE_1)$ and $PRE_1(m)$ is the circuit that outputs TRUE if the first bit of m is not 1. That is, ek_1 can only encapsulate messages that start with 1.
- $ek_0 = \mathsf{ACE.GenEK}(sk, PRE_0)$ and $PRE_0(m)$ is the circuit that outputs TRUE if the first bit of m is not 0. That is, ek_0 can only encapsulate messages that start with 0.

$\underline{\mathcal{D}_1^{(2)}}$

Hardcoded: cp^*, C^*, A, ek

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Sample $r_1^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$.
3. Set $pl_1 = 0||A||se||r_1^{**}||0^{\ell_4(\lambda)}$.
4. Set $x^* = \mathsf{ACE.Enc}(ek_0, pl_1)$.
5. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
6. Output $ch^*, (x^*, sp^*)$.

$\underline{\mathcal{D}_2^{(2)}}$

Hardcoded: cp^*, C^*, A, ek

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Sample $r_2^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$.
3. Set $pl_2 = 1||A||se||r_2^{**}||0^{\ell_4(\lambda)}$.
4. Set $x^* = \mathsf{ACE.Enc}(ek_1, pl_2)$.
5. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
6. Output $ch^*, (x^*, sp^*)$.

Finally, instead of applying $\mathsf{TI}^{(2)} \otimes \mathsf{TI}^{(2)}$ to the registers $\mathsf{R}_1, \mathsf{R}_2$, the challenger now applies $\mathsf{ATI}^{(3,1)} \otimes \mathsf{ATI}^{(3,2)}$ to the registers $\mathsf{R}_1, \mathsf{R}_2$, where $\mathsf{ATI}^{(3,1)} = \mathsf{ATI}^{\epsilon^*,\delta^*}_{\mathcal{D}_1^{(2)}, p^{\mathcal{F}}_{triv} + \frac{100\gamma}{128}}$ and $\mathsf{ATI}^{(3,2)} = \mathsf{ATI}^{\epsilon^*,\delta^*}_{\mathcal{D}_2^{(2)}, p^{\mathcal{F}}_{triv} + \frac{100\gamma}{128}}$. We still write $b^2_{Test,1}, b^2_{Test,2}$ to denote the measurement outcomes, respectively. Also, the challenger still outputs $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2}$ as the output of the game.

$\underline{\mathsf{Hyb}_7}$**:** The challenger flips a random coin $c' \leftarrow \{1,2\}$ and only applies $\mathsf{ATI}^{(3,c')}$ to $\mathsf{R}_{c'}$, instead of testing both registers. Let b^2_{Test} denote the measurement outcome. The challenger now outputs $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test}$ as the experiment outcome.

$\underline{\mathsf{Hyb}_8}$**:** After applying $\mathsf{ATI}^{(3,c')}$ to $\mathsf{R}_{c'}$, the challenger samples a challenge $ch^*, x^*, sp^* \leftarrow \mathcal{D}^{(2)}_{c'}$ and runs $y^* \leftarrow U(\mathsf{R}_{c'}, ch^*)$ and $b^2_{Ver} \leftarrow \mathsf{Ver}(sp^*, y^*)$. The challenger now outputs $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test}, b^2_{Ver}$ as the experiment outcome.

Hyb_9: The challenger now samples $\hat{M}$ as $\hat{M} \leftarrow i\mathcal{O}(M)$.

Hyb_{10}: We modify the challenger so that $\mathsf{TI}^{(2)} \otimes \mathsf{TI}^{(2)}$ and $\mathsf{ATI}^{(3,c')}$ are simulated using the punctured circuit C^{**} that is punctured as $C^*_{punc} \leftarrow \mathsf{Punc}(C^*, (ch^*, (x^*, sp^*)))$.

Hyb_{11}: We challenger checks if $b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1 \wedge b^2_{Test} = 1$. If not, it now sets y^* to be the trivial guess for ch^* instead of $y^* \leftarrow U(\mathsf{R}_{c'}, ch^*)$.

Claim 1. *There exists a polynomial $g(\cdot)$ such that $\Pr\left[b^2_{Ver} = 1\right] \geq \frac{1}{g(\lambda)}$ where $b^1_{Test,1}, b^1_{Test,2}, b^2_{Ver} \leftarrow \mathsf{Hyb}_{11}$.*

We prove this claim in Section 8.1

Lemma 26. $\Pr\left[b^2_{Ver} = 1\right] \leq \mathsf{negl}(\lambda)$ *where* $b^1_{Test,1}, b^1_{Test,2}, b^2_{Ver} \leftarrow \mathsf{Hyb}_{11}$.

Proof. This follows directly by Theorem 12. □

Since the statements above contradict each other, our hypothesis that there exists a QPT adversary $\mathcal{A}$ and polynomials $q_1(\lambda), q_2(\lambda)$ such that $\Pr\left[\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \geq \frac{1}{q_1(\lambda)}$ for infinitely many values of $\lambda > 0$ is wrong. Thus, for any polynomial $q_2(\lambda)$, we have $\Pr\left[\mathsf{StrongAntiPiracy}^{\gamma(\lambda)}_{\mathsf{QuantumProtect},\mathcal{A}}(1^\lambda) = 1\right] \leq \mathsf{negl}(\lambda)$ where $\gamma = \frac{1}{q_2(\lambda)}$, which completes the proof.

8.1 Indistinguishability of Hybrids: Proof of Claim 1

Claim 2. $\Pr\left[b^1_{Test} = 1\right] \geq \frac{3}{4q_1(\lambda)}$ *where* $b^1_{Test} \leftarrow \mathsf{Hyb}_1$.

Proof. Follows from Theorem 8. □

Claim 3. *For $b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}_2$,*

- $\Pr\left[b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{2q_1(\lambda)}$

Proof. This follows directly from Lemma 2. □

Claim 4. *For $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2} \leftarrow \mathsf{Hyb}_3$,*

- $\Pr\left[b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{2q_1(\lambda)}$
- $\Pr\left[b^2_{Test,1} \wedge b^2_{Test,2} = 1 \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq 1 - 6\delta^*$

Proof. Follows from Theorem 8. □

Claim 5. $\mathsf{Hyb}_3 \approx_{\alpha_3} \mathsf{Hyb}_4$

Proof. Observe that P and $P^{(1)}$ have exactly the same functionality: Due to the constrained decapsulation safety of ACE, $\mathsf{ACE.Dec}(dk', x)$ will always output $\perp$ since $dk' = \mathsf{ACE.GenDK}(ek, \mathsf{TRUE})$ (i.e, the key is punctured everywhere). Hence, the result follows by the security of $i\mathcal{O}$. □

Claim 6. $\mathsf{Hyb}_4 \approx_{\alpha_2} \mathsf{Hyb}_5$

Proof. Observe that the experiments do not use the encryption key ek at all, and they do not use any ACE ciphertexts either. Thus, the result follows by the puncture hiding security of ACE. □

Claim 7. *For* $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2} \leftarrow \mathsf{Hyb}_5$,

- $\Pr\left[b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{4q_1(\lambda)}$
- $\Pr\left[b^2_{Test,1} \wedge b^2_{Test,2} = 1 \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq 1 - 6\delta^* - 8q_1(\lambda) \cdot (\alpha_3 + \alpha_2)$

Proof. This follows by combining the claims above. □

Claim 8.

- $\Pr\left[b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{4q_1(\lambda)}$
- *There exists a polynomial* $g_2(\lambda)$ *such that*

$$\Pr\left[b^2_{Test,1} = 0 \wedge b^2_{Test,2} = 0 \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \leq 1 - \frac{1}{g_2(\lambda)}$$

where $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2} \leftarrow \mathsf{Hyb}_6$,

Proof. We prove this claim in Section 8.2. □

Claim 9. *For* $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test} \leftarrow \mathsf{Hyb}_7$,

- $\Pr\left[b^2_{Test} = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{8 \cdot q_1(\lambda) \cdot g_2(\lambda)}$

Proof. This follows directly from the claim above, since with probability $1/2$, c' will *hit* the *good* outcome between $b^2_{Test,1}, b^2_{Test,2}$. □

Claim 10. *For* $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test}, b^2_{Ver} \leftarrow \mathsf{Hyb}_8$,

- $\Pr\left[b^2_{Ver} = 1 \middle| b^2_{Test} = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq p^{\mathcal{F}}_{triv} + \frac{90\gamma}{128}$

Proof. Follows from Theorem 8. □

Claim 11. *For* $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test}, b^2_{Ver} \leftarrow \mathsf{Hyb}_9$,

- $\Pr\left[b^2_{Ver} = 1 \middle| b^2_{Test} = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq p^{\mathcal{F}}_{triv} + \frac{85\gamma}{128}$

Proof. Follows from Lemma 2. □

Claim 12. *For* $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test}, b^2_{Ver} \leftarrow \mathsf{Hyb}_{10}$,

- $\Pr\left[b^2_{Ver} = 1 \middle| b^2_{Test} = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq p^{\mathcal{F}}_{triv} + \frac{60\gamma}{128}$

Proof. Follows from weak resampling property of $\mathcal{F}$ and Lemma 6. □

Claim 13. *For* $b_{Ver} \leftarrow \mathsf{Hyb}_{11}$,

- $\Pr[b_{Ver} = 1] \geq p^{\mathcal{F}}_{triv} + \frac{60\gamma}{128} \cdot \frac{1}{64 \cdot q_1(\lambda) \cdot g_2(\lambda)}$

Proof. This follows since $\Pr\left[b_{Ver} = 1 | b^1_{Test,1} = b^1_{Test,2} = b^2_{Test} = 1\right] \geq p^{\mathcal{F}}_{triv} + \frac{60\gamma}{128}$ and $\Pr\left[b^1_{Test,1} = b^1_{Test,2} = b^2_{Test} = 1\right.$ $\left.\frac{1}{64 \cdot q_1(\lambda) \cdot g_2(\lambda)}\right.$ □

8.2 Reduction to Monogamy-of-Entanglement: Proof of Claim 8

The first item, that is $\Pr\left[b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq \frac{1}{4q_1(\lambda)}$, follows easily by the previous claim (since $b^1_{Test,1}=1, b^1_{Test,2}$ have the same marginal distribution in Hyb_5 and Hyb_4).

Now we prove the second item. Suppose for a contradiction that

$$\Pr\left[b^2_{Test,1}=0 \wedge b^2_{Test,2}=0 \middle| b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1-\mathsf{negl}(\lambda) \tag{1}$$

where $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2} \leftarrow \mathsf{Hyb}_5$. We will construct an adversary for Lemma 3.

Consider the following modified version Hyb^b_4 of Hyb_4: Simulate Hyb_4 until right after $\mathsf{ATI}^{(1)} \otimes \mathsf{ATI}^{(1)}$ is applied to $(\mathsf{R}_1, \mathsf{R}_2)$, but do not apply $\mathsf{TI}^{(2)} \otimes \mathsf{TI}^{(2)}$. Now, output $\mathsf{R}_1, \mathsf{R}_2, A, ek_b, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2}$ as the outcome of the experiment, where

- $ek_1 = \mathsf{ACE.GenEK}(sk, PRE_1)$ and $PRE_1(m)$ is the circuit that outputs TRUE if the first bit of m is not 1. That is, ek_1 can only encapsulate messages that start with 1.
- $ek_0 = \mathsf{ACE.GenEK}(sk, PRE_0)$ and $PRE_0(m)$ is the circuit that outputs TRUE if the first bit of m is not 0. That is, ek_0 can only encapsulate messages that start with 0.

We write Hyb'_4 to denote the version that outputs both ek_0 and ek_1.

Claim 14. *Let $\mathcal{M}$ be $\mathsf{ATI}^{(3,2)} = \mathsf{ATI}^{\epsilon^*,\delta^*}_{\mathcal{D}^{(2)}_2, p^{\mathcal{F}}_{triv}+\frac{100\gamma}{128}}$. Then,*

$$\Pr\left[\mathsf{TI}^{(2)}(\mathsf{R}_1)=1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1-3\cdot\sqrt{6\delta^*+8q_1(\lambda)\cdot(\alpha_3+\alpha_2)}$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$,

Proof. We already know

$$\Pr\left[(\mathsf{TI}^{(2)} \otimes \mathsf{TI}^{(2)})(\mathsf{R}_1, \mathsf{R}_2)=(1,1) \middle| b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1-6\delta^*-8q_1(\lambda)\cdot(\alpha_3+\alpha_2).$$

Then the result follows by Equation (1) and Theorem 5 since $\mathsf{TI}^{(2)}$ is a projective measurement. □

Claim 15. *Let $\mathcal{M}$ be $\mathsf{ATI}^{(3,2)} = \mathsf{ATI}^{\epsilon^*,\delta^*}_{\mathcal{D}^{(2)}_2, p^{\mathcal{F}}_{triv}+\frac{100\gamma}{128}}$. Then,*

$$\Pr\left[\mathsf{ATI}^{(3,1)}(\mathsf{R}_1)=0 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1-\mathsf{negl}(\lambda)$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$.

Claim 16. *Let $\mathcal{M}$ be a QPT algorithm with 1-bit output such that*

- $\Pr[\mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1)=0] \geq \frac{1}{g_3(\lambda)}$ *for some polynomial $g_3(\cdot)$*
- $\Pr\left[\mathsf{TI}^{(2)}(\mathsf{R}_1)=1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1-3\cdot\sqrt{+6\delta^*+8q_1(\lambda)\cdot(\alpha_3+\alpha_2)}$,
- $\Pr\left[\mathsf{ATI}^{(3,1)}(\mathsf{R}_1)=0 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1-\mathsf{negl}(\lambda)$.

Then, there exists an (unbounded) algorithm $\mathcal{E}^*_1$ such that

$$\Pr\left[\mathcal{E}^*_1(\mathsf{R}_1, A, ek_0, C^*, cp^*)=a^{Can}_1 \middle| b^1_{Test,1}=1 \wedge b^1_{Test,2}=1 \wedge \mathcal{M}(\mathsf{R}_2)=0\right] \geq 2^{-k}$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$.

We prove this claim in Section 8.2.1.

Claim 17. *Let $\mathcal{EV}_1$ be an algorithm with 1-bit output such that*

- *$\mathcal{EV}_1$ runs in subexponential time $rt(\lambda)$ (precise value will be clear from the proof)*
- $\Pr[\mathcal{EV}_1(\mathsf{R}_1, A, ek_0, C^*, cp^*) = 1] \geq 2^{-k}$
- $$\Pr\left[\mathsf{TI}^{(2)}(\mathsf{R}_2) = 1 \middle| \mathcal{EV}_1(\mathsf{R}_1, A, ek_0, C^*, cp^*, a_1^{Can}) = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq 1 - \frac{3}{2} \cdot \sqrt{6\delta^* + 8q_1(\lambda) \cdot (\alpha_3 + \alpha_2)} \cdot \frac{2}{2^{-k}},$$
- $\Pr\left[\mathsf{ATI}^{(3,2)}(\mathsf{R}_2) = 0 \middle| \mathcal{EV}_1(\mathsf{R}_1, A, ek_0, C^*, cp^*, a_1^{Can}) = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{4}2^{-k}$

Then, there exists an (unbounded) algorithm $\mathcal{E}_2^*$ such that

$$\Pr\left[\mathcal{E}_2^*(\mathsf{R}_2, A, ek_1, C^*, cp^*) = a_2^{Can} \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1 \wedge \mathcal{EV}_1(\mathsf{R}_1, A, ek_0, C^*, cp^*) = 1\right] \geq 2^{-k}$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$.

We prove this claim in Section 8.2.1.

Claim 18. *There exists unbounded algorithms $\mathcal{E}_1^*, \mathcal{E}_2^*$ such that*

$$\Pr\left[\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) = a_1^{Can} \wedge \mathcal{E}_2^*(\mathsf{R}_2, A, ek_1, C^*, cp^*) = a_2^{Can} \middle| b^1_{Test,1} = 1 \wedge b^2_{Test,2} = 1\right] \geq (2^{-k})^2 \cdot \frac{1}{2}.$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$.

Proof. Set $\mathcal{M} = \mathsf{ATI}^{(3,2)}$. First, by Claim 14,Claim 15, and Claim 16, we have that there exist $\mathcal{E}_1^*$ such that

$$p_1 = \Pr\left[\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) = a_1^{Can} \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1 \wedge \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0\right] \geq 2^{-k}$$

which also implies

$$p_1 = \Pr\left[\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) = a_1^{Can} \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq 2^{-k} \cdot \frac{1}{2}$$

Now, let $\mathcal{EV}_1(1^\lambda)$ be an algorithm simply simulates the output distribution of the equality check $\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) \stackrel{?}{=} a_1^{Can}$. That is, $\mathcal{EV}_1(1^\lambda)$ simulates a biased coin and outputs 1 with probability p_1. Now we claim the following.

- $$\Pr\left[\mathsf{TI}^{(2)}(\mathsf{R}_2) = 1 \middle| \mathcal{EV}(\mathsf{R}_1, A, ek_0, C^*, cp^*, a_1^{Can}) = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq 1 - \frac{3}{2} \cdot \sqrt{6\delta^* + 8q_1(\lambda) \cdot (\alpha_3 + \alpha_2)} \cdot \frac{2}{2^{-k}},$$
- $\Pr\left[\mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \middle| \mathcal{EV}(\mathsf{R}_1, A, ek_0, C^*, cp^*, a_1^{Can}) = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \frac{1}{4}2^{-k}$

For the first claim, we already know

$$\Pr\Big[(\mathsf{TI}^{(2)} \otimes \mathsf{TI}^{(2)})(\mathsf{R}_1, \mathsf{R}_2) = (1,1) \big| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big] \geq 1 - 6\delta^* - 8q_1(\lambda) \cdot (\alpha_3 + \alpha_2).$$

Then the first claim follows by Theorem 5 since $\Pr\Big[\mathcal{EV}((\mathsf{R}_1, A, ek_0, C^*, cp^*), a_1^{Can}) = 1 \big| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big] \geq 2^{-k}$ and $\mathsf{TI}^{(2)}$ is a projective measurement.

To see the second claim, first we have by Bayes' Theorem that,

$$\frac{\Pr\Big[\mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \big| \mathcal{EV}(\mathsf{R}_1, A, ek_0, C^*, cp^*, a_1^{Can}) = 1 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big]}{\Pr\Big[\mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 = 0 \big| \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big]} \geq$$
$$\Pr\big[\mathcal{EV}(\mathsf{R}_1, A, ek_0, C^*, cp^*, a_1^{Can}) = 1 \big| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\big]$$

The result then follows, since $\Pr\Big[\mathcal{EV}((\mathsf{R}_1, A, ek_0, C^*, cp^*), a_1^{Can}) = 1 \big| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big] \geq \frac{1}{2}2^{-k}$ by above and $\Pr\Big[\mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 = 0\big| \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big] \geq \frac{1}{2}$ by Equation (1).

Then, applying Claim 17 with $\mathcal{EV}_1(1^\lambda)$, we get

$$\Pr\big[\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) = a_1^{Can} \wedge \mathcal{E}_2^*(\mathsf{R}_2, A, ek_1, C^*, cp^*) = a_2^{Can} \big| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1 \wedge \mathcal{M}(\mathsf{R}_2) = 0\big] \geq (2^{-k})^2$$

which completes the proof. □

Since $\Pr\Big[b^1_{Test,1} = 1 \wedge b^2_{Test,2} = 1\Big] \geq \frac{1}{4q_1(\lambda)}$, the claim above is a contradiction to Lemma 3 by our choice of parameters. Thus, our contradiction hypothesis that

$$\Pr\big[b^2_{Test,1} = 0 \wedge b^2_{Test,2} = 0 \big| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\big] \geq 1 - \mathsf{negl}(\lambda),$$

where $b^1_{Test,1}, b^1_{Test,2}, b^2_{Test,1}, b^2_{Test,2} \leftarrow \mathsf{Hyb}_5$, is incorrect. This proves Claim 8.

8.2.1 Proof of Claim 16

We will instead prove the following stronger claim, so that the proof can serve as a proof for both[25] Claim 16 and Claim 17.

Claim 19. *Let $\mathcal{M}$ be an algorithm with 1-bit output such that*

- *$\mathcal{M}$ runs in subexponential time $rt(\lambda)$ (precise value will be clear from the proof)*
- $\Pr[\mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0] \geq 2^{-k}$,
- $\Pr\Big[\mathsf{TI}^{(2)}(\mathsf{R}_1) = 1 \big| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big] \geq 1 - \frac{3}{2} \cdot \sqrt{6\delta^* + 8q_1(\lambda) \cdot (\alpha_3 + \alpha_2)} \cdot \frac{2}{2^{-k}}$,
- $\Pr\Big[\mathsf{ATI}^{(3,1)}(\mathsf{R}_1) = 0 \big| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\Big] \geq \frac{1}{4}2^{-k}$.

Then, there exists an (unbounded) algorithm $\mathcal{E}_1^*$ such that

$$\Pr\big[\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) = a_1^{Can} \big| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1 \wedge \mathcal{M}(\mathsf{R}_2) = 0\big] \geq 2^{-k}$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$.

[25] Note that while technically Claim 17 is not a consequence of our stronger claim we prove in this section, proof of Claim 17 follows almost exactly the same as the proof of this stronger version of Claim 16.

We now prove this claim. Let $\mathcal{M}$ be an algorithm as in the claim statement. Define $\mathsf{SimATI}^{(4)} = \mathsf{SimATI}^{\epsilon^*,\delta^*,\alpha_1(\lambda)}_{|\max\{\mathcal{D}(C^*,cp^*),\mathcal{D}_1^{(2)}\}|,\frac{116\gamma}{128}}$ and let $t_1(\lambda)$ be the number of samples for $\mathsf{SimATI}^{(4)}$ as in Lemma 6. Then, by Lemma 6, we get

- $\Pr\left[\mathsf{SimATI}^{(4)}(s_{1,1},\ldots,s_{1,t_1(\lambda)})(\mathsf{R}_1)=1 \middle| \mathcal{M}(\mathsf{R}_2,C^*,cp^*,ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq 1 - \frac{3}{2}\cdot\sqrt{6\delta^*+8q_1(\lambda)\cdot(\alpha_3+\alpha_2)}\cdot\frac{2}{2^{-k}} - \alpha_1 - 4\delta^* = \xi_1$,
- $\Pr\left[\mathsf{SimATI}^{(4)}(s_{2,1},\ldots,s_{2,t_1(\lambda)})(\mathsf{R}_1)=0 \middle| \mathcal{M}(\mathsf{R}_2,C^*,cp^*,ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq \frac{1}{4}2^{-k} - \alpha_1 - 4\delta^*$.

where $s_{1,1},\ldots,s_{1,t_1(\lambda)} \leftarrow \mathcal{D}(cp^*,C^*)$ and $s_{2,1},\ldots,s_{2,t_1(\lambda)} \leftarrow \mathcal{D}_1^{(2)}$.

Now define the following distributions for $j \in [t_1(\lambda)]$.

$\underline{\mathcal{D}_1^{(2,j)}}$

Hardcoded: cp^*, C^*, A, ek

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Sample $r_1^{**} \leftarrow \{0,1\}^{p_4(\lambda)}$.
3. Set $pl_1 = 0||A||se||r_1^{**}||j$.
4. Set $x^* = \mathsf{ACE.Enc}(ek_0, pl_1)$.
5. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
6. Output $ch^*, (x^*, sp^*)$.

Now we claim the following.

Claim 20. $\Pr\left[\mathsf{SimATI}^{(4)}(s_{2,1},\ldots,s_{2,t_1(\lambda)})(\mathsf{R}_1)=0 \middle| \mathcal{M}(\mathsf{R}_2,C^*,cp^*,ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq \frac{1}{4}2^{-k} - \alpha_1 - 4\delta^* - ((\alpha_2+2\alpha_3)\cdot t_1(\lambda) - 2\alpha_3)\cdot\frac{1}{2^{-k}} = \xi_2$ *where* $s_{2,j} \leftarrow \mathcal{D}_1^{(2,j)}$ *for* $j \in [t_1(\lambda)]$

Proof. This claim follows easily by an ACE key puncturing and iO argument, changing each sample one by one, since the program ignores the *ind* part of the decrypted messages. □

By applying the hybrid lemma to above, we get that there exists some index $i^* \in \{1,\ldots,t_1(\lambda)\}$ and values τ_1, τ_2 such that

- $\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1)=1 \middle| \mathcal{M}(\mathsf{R}_2,C^*,cp^*,ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq \tau_1$,
- $\Pr\left[\mathsf{SimATI}^{(4)}((sr_i)_{i\in[t(\lambda)]})(\mathsf{R}_1)=0 \middle| \mathcal{M}(\mathsf{R}_2,C^*,cp^*,ek_1)=0 \wedge b^1_{Test,1}=1 \wedge b^1_{Test,2}=1\right] \geq \tau_2$
- $\tau_2 - \tau_1 \geq \frac{\xi_2-\xi_1}{t_1(\lambda)}$

where

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}''_4$ and
- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i < i^*$ and $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ otherwise,
- $sr_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i \leq i^*$ and $sr_i \leftarrow \mathcal{D}(cp^*, C^*)$ otherwise.

Then we define the following distribution and claim the following.

$\underline{\mathcal{D}_1^{(1,j)}}$

Hardcoded: $cp^*, C^*, A, a_1^{Can}, ek, r_1^*$

1. Sample an extractor seed as $se \leftarrow \{0,1\}^{\ell_3(\lambda)}$.
2. Set $pl_1 = 0||A||se||(\mathsf{Ext}(se, a_1^{Can}) \oplus r_1^*)||j$
3. Set $x^* = \mathsf{ACE.Enc}(ek_0, pl_1)$.
4. Sample $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{chal}}(cp^*, C^*, x^*)$.
5. Output $ch^*, (x^*, sp^*)$.

Claim 21.
$\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1) = 1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \tau_1 - (2\alpha_2 + \alpha_3 + 2\alpha_4) \cdot \frac{4}{2^{-k}}$ *where*

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$
- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ *for* $i < i^*$
- $sl_i \leftarrow \mathcal{D}_1^{(1,i)}$ *if* $i = i^*$,
- $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ *for* $i > i^*$,

Proof. We will define a sequence of modifications for Hyb'_4.

We define the following modification $\mathsf{Hyb}_4^{'(1)}$ of Hyb'_4: The challenger samples $\hat{M}$ as $\hat{M} \leftarrow i\mathcal{O}(M)$ instead of $\hat{M} \leftarrow i\mathcal{O}(M')$. This essentially undoes the change from Hyb_1 to Hyb_2.

By Lemma 2, we have $\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1) = 1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \tau_1 - (\alpha_4) \cdot \frac{4}{2^{-k}}$ where

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}_4^{'(1)}$
- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i < i^*$
- $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ for $i \geq i^*$,

We define $\mathsf{Hyb}_4^{'(2)}$ so that when the challenger is preparing $\hat{P}$, it now embeds $dk' = \mathsf{ACE.GenDK}(sk, T_{pl^*})$ instead of $dk' = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$ where

- $T_{pl^*}(m)$ is the circuit that outputs TRUE if $m = pl^*$,
- $pl^* = \mathsf{ACE.Enc}(ek_0, 0||A||se^*||(\mathsf{Ext}(se^*, a_1^{Can}) \oplus r_1^*)||i^*)$ where se^*, r_1^* are the values sampled during sampling $sl_{i^*} \leftarrow \mathcal{D}_1^{(1,i^*)}$

Since the experiments do not use an encryption of pl^*, we can apply the puncture-hiding security of ACE to get $\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1) = 1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \tau_1 - (\alpha_2 + \alpha_4) \cdot \frac{4}{2^{-k}}$ where

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}_4^{'(2)}$

- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i < i^*$
- $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ for $i \geq i^*$,

Now, we claim $\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1) = 1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \tau_1 - (2\alpha_2 + \alpha_4) \cdot \frac{4}{2^{-k}}$ where

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}_4^{'(2)}$
- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i < i^*$
- $sl_i \leftarrow \mathcal{D}_1^{(1,i)}$ if $i = i^*$,
- $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ for $i > i^*$,

This follows by pseudorandom ciphertext property of ACE since dk' is punctured at pl^*.

We define $\mathsf{Hyb}_4^{'(3)}$ so that when the challenger is preparing $\hat{P}$, it now embeds $dk' = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$. We claim $\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1) = 1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \tau_1 - (\alpha_3 + \alpha_4 + 2\alpha_2) \cdot \frac{4}{2^{-k}}$ where

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}_4^{'(3)}$
- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i < i^*$
- $sl_i \leftarrow \mathcal{D}_1^{(1,i)}$ if $i = i^*$,
- $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ for $i > i^*$,

This follows by the security of $i\mathcal{O}$ since by construction of pl^*, the programs have the same functionality in both cases.

Finally, we claim $\Pr\left[\mathsf{SimATI}^{(4)}((sl_i)_{i\in[t(\lambda)]})(\mathsf{R}_1) = 1 \middle| \mathcal{M}(\mathsf{R}_2, C^*, cp^*, ek_1) = 0 \wedge b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1\right] \geq \tau_1 - (\alpha_3 + 2\alpha_4 + 2\alpha_2) \cdot \frac{4}{2^{-k}}$ where

- $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}_4^{'(3)}$
- $sl_i \leftarrow \mathcal{D}_1^{(2,i)}$ for $i < i^*$
- $sl_i \leftarrow \mathcal{D}_1^{(1,i)}$ if $i = i^*$,
- $sl_i \leftarrow \mathcal{D}(cp^*, C^*)$ for $i > i^*$,

This follows directly by Lemma 2. □

The claim above shows that we can distinguish between $\mathcal{D}_1^{(1,i^*)}$ and $\mathcal{D}_1^{(2,i^*)}$ with advantage $\geq \frac{\xi_2 - \xi_1}{t_1(\lambda)} - \tau_1 - (2\alpha_2 + \alpha_3 + 2\alpha_4) \cdot \frac{4}{2^{-k}}$. Since this is larger than ϵ_{Ext} by our choice of parameters, we get by the reconstructive property of Ext that there exists an algorithm $\mathcal{E}_1^*$ such that

$$\Pr\left[\mathcal{E}_1^*(\mathsf{R}_1, A, ek_0, C^*, cp^*) = a_1^{Can} \middle| b^1_{Test,1} = 1 \wedge b^1_{Test,2} = 1 \wedge \mathcal{M}(\mathsf{R}_2) = 0\right] \geq 2^{-k}$$

where $\mathsf{R}_1, \mathsf{R}_2, A, ek_0, ek_1, C^*, cp^*, b^1_{Test,1}, b^1_{Test,2} \leftarrow \mathsf{Hyb}'_4$.

This completes the proof.

9 The Common Last Hybrid

We define the following security game between an adversary $\mathcal{A}$ and a challenger. This game essentially captures the final hybrids in all three quantum protection security proofs we have in this work.

Let $\mathcal{F} = (\Phi, \mathsf{Eval}, \mathsf{Chal}, \mathcal{D}, \mathsf{Ver}, \mathsf{Punc})$ be a puncturable functionality (Definition 5).

$\underline{\mathsf{QPPuncGame}_{\mathcal{A}}(1^\lambda)}$

1. $\mathcal{A}$ submits a bit $c' \in \{1, 2\}$.
2. Chal and $\mathcal{A}$ interact, Chal outputs C^* and cp^*.
3. The challenger samples $(\hat{M}, \mathsf{R}') \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$. Let A denote the subspace sampled during this sampling.
4. The challenger samples $K \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.
5. The challenger samples $r_1^*, r_2^* \leftarrow \{0,1\}^{p_4(\lambda)}$ and computes $ct_1^* = C^* \oplus G(r_1^*), ct_2^* = K \oplus G(r_2^*)$.
6. The challenger samples $se^* \leftarrow \{0,1\}^{\ell_3(\lambda)}$ and r^{**}.
7. The challenger computes $pl^* = 0||A||se^*||r^{**}||0^{\ell_4(\lambda)}$ if $c' = 1$. If $c' = 2$, it computes $pl^* = 1||A||se^*||r^{**}||0^{\ell_4(\lambda)}$.
8. The challenger samples ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$ and computes $ek = \mathsf{ACE.GenEK}(sk, \mathbb{1}_{pl^*})$ and $dk = \mathsf{ACE.GenEK}(sk, \mathsf{FALSE})$.
9. The challenger samples $\hat{P} \leftarrow i\mathcal{O}(P')$, where $i\mathcal{O}(P')$ is the following program.

$P'(x,b,v)$

Hardcoded: $\hat{M}, K, C^*, dk, ct_1^*, ct_2^*$

1. Check if $\hat{M}(b,v) = \mathsf{TRUE}$. If not, output $\bot$ and terminate.
2. Compute $pl = \mathsf{ACE.Dec}(dk, x)$. If $pl \neq \bot$, parse $t||A'||se'||z||ind = pl$.
3. If $pl = \bot$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
   1. If $b = 0$, output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$ and terminate.
4. If $pl \neq \bot$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A', v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}((A')^{\bot}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $K' = ct_2^* \oplus G(r')$.
      3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

10. The challenger computes $act^* = \mathsf{ACE.Enc}(ek, pl^*)$.
11. The challenger samples $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{ch}}(cp^*, C^*, act^*)$.
12. The challenger samples $C^*_{punc} \leftarrow \mathsf{Punc}(C^*, (ch^*, (act^*, sp^*)))$.
13. $\mathcal{A}$ receives $\hat{P}, ek', ch^*, cp^*, C^*_{punc}, act^*, \mathsf{R}'$.
14. For any polynomial number of times: The adversary $\mathcal{A}$ submits a sample request, and the challenger replies by sampling $ch, (x, sp') \leftarrow \mathcal{D}(cp^*, C^*)$ and submitting $ch, (x, sp')$ to the adversary.
15. $\mathcal{A}$ outputs y^*.
16. The challenger outputs 1 if $\mathsf{Ver}((act^*, sp^*), y^*) = \mathsf{TRUE}$, otherwise, it outputs 0.

We claim the following.

Theorem 12. *For any QPT adversary $\mathcal{A}$,*

$$\Pr\left[\mathsf{QPPuncGame}_{\mathcal{A}}(1^\lambda) = 1\right] \leq p^{\mathcal{F}}_{triv} + \mathsf{negl}(\lambda).$$

9.1 Proof of Theorem 12

In this section, we prove Theorem 12. Our proof will proceed through a sequence of hybrids, each of which is constructed by modifying the previous one.

Let $\mathcal{A}$ be any QPT adversary.

Hyb_0**:** The original game $\mathsf{QPPuncGame}_{\mathcal{A}}(1^\lambda)$.

$\underline{\mathsf{Hyb}_1}$**:** First, we compute z^* as follows. If $c' = 1$, we compute $C' = ct_1^* \oplus G(\mathsf{Ext}(se^*, a_1^{Can}) \oplus r^{**})$ and $z^* = C'(act^*) \oplus F(K, act^*)$ where C' is interpreted as a circuit. If $c' = 2$, we compute $z^* = \mathsf{PRF.Eval}(K', act^*)$ where $K' = ct_2^* \oplus G(\mathsf{Ext}(se^*, a_2^{Can}) \oplus r^{**})$.

Further, we now set $dk' = \mathsf{ACE.GenDK}(sk, \mathbb{1}_{pl^*})$.

Then, we modify the obfuscated program $\hat{P}$ by now sampling it as $\hat{P} \leftarrow i\mathcal{O}(P^{(2,c')})$ where $P^{(2,1)}$ and $P^{(2,2)}$ are defined below.

$\underline{P^{(2,1)}(x, b, v)}$

Hardcoded: $\hat{M}, K, C^*, ct_1^*, ct_2^*, z^*, act^*, dk'$

1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\perp$ and terminate.
2. If $x = act^*$,
   1. If $b = 0$, output z^* and terminate.
   2. Otherwise (if $b = 1$), output $\mathsf{PRF.Eval}(K, x)$ and terminate.
3. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \perp$, parse $t||A'||se'||z||ind = pl'$.
4. If $pl = \perp$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
   1. If $b = 0$, output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$ and terminate.
5. If $pl \neq \perp$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A', v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}((A')^{\perp}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $K' = ct_2^* \oplus G(r')$.
      3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

$\underline{P^{(2,2)}(x, b, v)}$

Hardcoded: $\hat{M}, K, C^*, ct_1^*, ct_2^*, z^*, act^*, dk'$

1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\perp$ and terminate.
2. If $x = act^*$,
   1. If $b = 1$, output z^* and terminate.
   2. Otherwise (if $b = 0$), output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
3. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \perp$, parse $t||A'||se'||z||ind = pl$.
4. If $pl = \perp$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,

1. If $b = 0$, output $C^*(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
2. If $b = 1$, output $\mathsf{PRF.Eval}(K, x)$ and terminate.

5. If $pl \neq \bot$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}(A^{\bot}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $K' = ct_2^* \oplus G(r')$.
      3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

Hyb_2**:** The challenger now samples $act^* \leftarrow \mathcal{D}_{\mathsf{inp}}(cp^*, C^*)$ instead of $act^* = \mathsf{ACE.Enc}(ek, pl^*)$.

Hyb_3**:** We sample $K\{act^*\} \leftarrow \mathsf{PRF.Punc}(K, act^*)$. We also set $s_1^* = \mathsf{PRF.Eval}(K, act^*)$ and $s_2^* = C^*(act^*) \oplus s_1^*$. Then, we modify the obfuscated program $\hat{P}$ by now sampling it as $\hat{P} \leftarrow i\mathcal{O}(P^{(3,c')})$.

$P^{(3,1)}(x, b, v)$

Hardcoded: $\hat{M}, K\{act^*\}, C^*_{punc}, s_1^*, ct_1^*, ct_2^*, z^*, act^*, dk'$

1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\bot$ and terminate.
2. If $x = act^*$,
   1. If $b = 0$, output z^* and terminate.
   2. Otherwise (if $b = 1$), output s_1^* and terminate.
3. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \bot$, parse $t||A'||se'||z||ind = pl$.
4. If $pl = \bot$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
   1. If $b = 0$, output $C^*_{punc}(x) \oplus \mathsf{PRF.Eval}(K\{act^*\}, x)$ and terminate.
   2. If $b = 1$, output $\mathsf{PRF.Eval}(K\{act^*\}, x)$ and terminate.
5. If $pl \neq \bot$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A', v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}((A')^{\bot}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.

2. Compute $K' = ct_2^* \oplus G(r')$.
3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

$\underline{P^{(3,2)}(x, b, v)}$

Hardcoded: $\hat{M}, K\{act^*\}, C^*_{punc}, s_2^*, ct_1^*, ct_2^*, z^*, act^*, dk'$

1. Check if $\hat{M}(b, v) = \mathsf{TRUE}$. If not, output $\perp$ and terminate.
2. If $x = act^*$,
   1. If $b = 1$, output z^* and terminate.
   2. Otherwise (if $b = 0$), output s_2^* and terminate.
3. Compute $pl = \mathsf{ACE.Dec}(dk', x)$. If $pl \neq \perp$, parse $t||A'||se'||z||ind = pl$.
4. If $pl = \perp$ or if $b = 0 \wedge t = 1$ or if $b = 1 \wedge t = 0$,
   1. If $b = 0$, output $C^*_{punc}(x) \oplus \mathsf{PRF.Eval}(K\{act^*\}, x)$ and terminate.
   2. If $b = 1$, output $\mathsf{PRF.Eval}(K\{act^*\}, x)$ and terminate.
5. If $pl \neq \perp$,
   1. If $t = 0$,
      1. Compute $a' = \mathsf{Can}(A, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $C' = ct_1^* \oplus G(r')$.
      3. Output $C'(x) \oplus \mathsf{PRF.Eval}(K, x)$ and terminate.
   2. If $t = 1$,
      1. Compute $a' = \mathsf{Can}(A^{\perp}, v)$ and $r' = \mathsf{Ext}(se', a') \oplus z$.
      2. Compute $K' = ct_2^* \oplus G(r')$.
      3. Output $\mathsf{PRF.Eval}(K', x)$ and terminate.

$\underline{\mathsf{Hyb}_4}$**:** We modify the way ct_1^*, ct_2^* are computed: Now we sample $ct_1^*, ct_2^* \leftarrow \{0,1\}^{p_2(\lambda)}$.

$\underline{\mathsf{Hyb}_5}$**:** If $c' = 1$, we sample s_1^* uniformly at random, and also set $z^* = C'(act^*) \oplus s_1^*$ where $C' = ct_1^* \oplus G(\mathsf{Ext}(se^*, a_1^{Can}) \oplus r^{**})$, and also set $s_2^* = \perp$. If $c' = 2$, we sample s_2^* uniformly at random and set $s_1^* = \perp$. Also recall that when $c' = 2$, z^* is computed as $z^* = \mathsf{PRF.Eval}(K', act^*)$ where $K' = ct_2^* \oplus G(\mathsf{Ext}(se^*, a_2^{Can}) \oplus r^{**})$, which does not use the *actual* PRF key K.

$\underline{\mathsf{Hyb}_6}$**:** We modify the way the challenger answers the queries of $\mathcal{A}$ at the last step. Instead of sampling $ch, (x, sp) \leftarrow \mathcal{D}(cp^*, C^*)$, the challenger now samples $ch, (x, sp) \leftarrow \mathcal{D}(cp^*, C^*_{punc})$ and submits $ch, (x, sp)$ to the adversary.

$\underline{\mathsf{Hyb}_7}$**:** We define Hyb_7 by reordering some independent steps (e.g. sampling of independent values) of Hyb_6. We write the full experiment Hyb_7 below for ease of reference.

Hyb_7

1. Chal and $\mathcal{A}$ interact, Chal outputs C^* and cp^*.
2. The challenger samples $x^* \leftarrow \mathcal{D}_{\mathsf{inp}}(cp^*, C^*)$.
3. The challenger samples $ch^*, sp^* \leftarrow \mathcal{D}_{\mathsf{ch}}(cp^*, C^*, x^*)$.
4. The challenger samples $C^*_{punc} \leftarrow \mathsf{Punc}(C^*, (ch^*, (x^*, sp^*))$.
5. The challenger samples $ct^*_1, ct^*_2 \leftarrow \{0,1\}^{p_2(\lambda)}$.
6. The challenger samples $(\hat{M}, \mathsf{R}') \leftarrow \mathsf{QuantumProtect.GenState}(1^\lambda)$. Let A denote the subspace sampled during this sampling.
7. The challenger samples $se^* \leftarrow \{0,1\}^{\ell_3(\lambda)}$ and r^{**}.
8. The challenger samples $r^*_1, r^*_2 \leftarrow \{0,1\}^{p_4(\lambda)}$.
9. $\mathcal{A}$ submits a bit $c' \in \{1,2\}$.
10. The challenger computes $pl^* = 0||A||se^*||r^{**}||0^{\ell_4(\lambda)}$ if $c' = 1$. If $c' = 2$, it computes $pl^* = 1||A||se^*||r^{**}||0^{\ell_4(\lambda)}$.
11. If $c' = 1$, we sample s^*_1 uniformly at random, and also set $z^* = C'(x^*) \oplus s^*_1$ where $C' = ct^*_1 \oplus G(\mathsf{Ext}(se^*, a^{Can}_1) \oplus r^{**})$, and also set $s^*_2 = \perp$. If $c' = 2$, we sample s^*_2 uniformly at random and set $s^*_1 = \perp$, and compute $z^* = \mathsf{PRF.Eval}(K', x^*)$ where $K' = ct^*_2 \oplus G(\mathsf{Ext}(se^*, a^{Can}_2) \oplus r^{**})$.
12. The challenger samples $K \leftarrow \mathsf{PRF.Setup}(1^\lambda)$ and $K\{x^*\} \leftarrow \mathsf{PRF.Punc}(K, x^*)$
13. The challenger samples ACE instance as $sk \leftarrow \mathsf{ACE.Setup}(1^\lambda)$ and computes $ek' = \mathsf{ACE.GenEK}(sk, \mathbb{1}_{pl^*})$ and $dk' = \mathsf{ACE.GenEK}(sk, \mathbb{1}_{pl^*})$.
14. The challenger samples $\hat{P} \leftarrow i\mathcal{O}(P^{(3,c')})$ which uses $\hat{M}, K\{x^*\}, C^*_{punc}, dk', ct^*_1, ct^*_2, z^*, x^*$ and s^*_1 if $c' = 1$ or s^*_2 if $c' = 2$.
15. $\mathcal{A}$ receives $\hat{P}, ek', ch^*, cp^*, C^*_{punc}, x^*, A, a_1, a_2, \mathsf{R}'$.
16. For any polynomial number of times: The adversary $\mathcal{A}$ submits a sample request, and the challenger replies by sampling $ch, (x, sp') \leftarrow \mathcal{D}(cp^*, C^*_{punc})$ and submitting $ch, (x, sp')$ to the adversary.
17. $\mathcal{A}$ outputs y^*.
18. The challenger outputs 1 if $\mathsf{Ver}(sp^*, y^*) = \mathsf{TRUE}$, otherwise, it outputs 0.

Lemma 27. $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_7$.

We prove this lemma in Section 9.1.1.

Lemma 28. $\Pr[\mathsf{Hyb}_7 = 1] \leq \mathsf{negl}(\lambda)$.

Proof. We will prove this lemma using the puncturing security of $\mathcal{F}$.

Consider the following adversary $\mathcal{B}$ for the puncturing security game of $\mathcal{F}$.

$\underline{\mathcal{B}(1^\lambda)}$

1. **Setup Phase:** Simulate $\mathcal{A}$ to interact with Chal.
2. **Challenge Phase:** Receive $cp^*, ch^*, C^*_{punc}, x^*$ from the challenger.
3. Simulate Hyb_7 (both *the challenger* and the adversary steps, that is, all the steps) from Item 5 to Item 16 (both steps included).
4. Output y^*.

First observe that the steps from Item 5 to Item 16 of Hyb_7 (both *the challenger* and the adversary steps) indeed only uses $cp^*, ch^*, C^*_{punc}, x^*$, thus, $\mathcal{B}$ is a valid adversary for the puncturing game of $\mathcal{F}$. It is also easy to see that $\Pr\left[\mathsf{PuncturingGame}_{\mathcal{B}}(1^\lambda)\right] = \Pr[\mathsf{Hyb}_7 = 1]$. Thus, the result follows by the puncturing security of $\mathcal{F}$. □

Finally, we complete the proof of Theorem 12. Suppose for a contradiction that there exists a QPT adversary $\mathcal{A}$ and a polynomial $q(\cdot)$ such that $\Pr\left[\mathsf{QPPuncGame}_{\mathcal{A}}(1^\lambda) = 1\right] \geq \frac{1}{q(\lambda)}$ for an infinite number of values $\lambda > 0$. Then, by Lemma 27, we get $\Pr[\mathsf{Hyb}_7 = 1] \geq \frac{1}{2q(\lambda)}$ for an infinite number of values $\lambda > 0$. However, this is a contradiction to Lemma 28.

9.1.1 Indistinguishability of Hybrids: Proof of Lemma 27

Lemma 29. $\mathsf{Hyb}_0 \approx \mathsf{Hyb}_1$.

Proof. We will prove the two cases $c' = 1$ and $c' = 2$ separately.

First, the case $c' = 1$. We claim that $P^{'}$ and $P^{(2,1)}$ have exactly the same functionality. Note that once we prove this, the result follows by $i\mathcal{O}$ security. Now we prove our claim. Observe that $P^{'}$ and $P^{(2,1)}$ can differ in two places: (i) when using the decryption key dk' in the line $pl = \mathsf{ACE.Dec}(dk', x)$, since $P^{'}$ has $dk = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$ but $P^{(2,1)}$ has $dk' = \mathsf{ACE.GenDK}(sk, \mathbb{1}_{pl^*})$, and (ii) if the input satisfies $x = act^*$. First, by the safety of constrained decapsulation of ACE, we know that the output of $\mathsf{ACE.Dec}$ on x can be different for $dk = \mathsf{ACE.GenDK}(sk, \mathsf{FALSE})$ versus $dk' = \mathsf{ACE.GenDK}(sk, \mathbb{1}_{pl^*})$ only if $x = \mathsf{ACE.Enc}(ek, pl^*) = act^*$. Thus, we get that $P^{'}$ and $P^{(2,1)}$ can possibly differ only on inputs such that $x = act^*$. Further it easy to see that $P^{'}(x, b, v)$ versus $P^{(2,1)}(x, b, v)$ can only possibly differ if $\hat{M}(b, v) = \mathsf{TRUE}$ is satisfied too. Thus, we only need to consider inputs x, b, v such that $x = act^*$ and $\hat{M}(b, v) = \mathsf{TRUE}$. For such inputs, we will further consider two cases: $b = 0$ and $b = 1$. For $b = 1$, it is easy to see that $P^{'}(x, b, v) = \mathsf{PRF.Eval}(K, act^*) = P^{(2,1)}(x, b, v)$. Finally, we consider the case $b = 0$. Now, $P^{'}(x, b, v)$ enters the case $pl \neq \perp, t = 0$ since act^* decrypts to pl^* by correctness of decapsulation. Here, it gets $a' = a_1^{Can}$ since $v \in A + a_1$, and then it gets $r' = \mathsf{Ext}(se^*, a_1^{Can}) \oplus r^{**}$ since $pl = pl^*$. Then, it gets $C' = ct_1 \oplus G(\mathsf{Ext}(se^*, a_1^{Can}) \oplus r^{**})$. Thus, we get $P^{'}(x, b, v) = z^*$. Now considering $P^{(2,1)}(x, b, v)$ when $b = 0$, $x = act^*$ and $v \in A + s$, it is easy to see that we again have $P^{(2,1)}(x, b, v) = z^*$. Thus, $P^{(1)}$ and $P^{(2,1)}$ have the same output for all x, b, v.

Now we prove the case $c' = 2$. We claim that $P^{(')}$ and $P^{(2,2)}$ have exactly the same functionality. Similar to above, once we prove this, the result follows by $i\mathcal{O}$ security. The claim that $P^{'}$ and $P^{(2,2)}$ have the same functionality follows by a case-by-case inspection, similar to above. □

Lemma 30. $\mathsf{Hyb}_1 \approx \mathsf{Hyb}_2$.

Proof. Observe that these experiments only use the constrained keys $dk' = \mathsf{ACE.GenDK}(sk, \mathbb{1}_{pl^*})$ and $ek' = \mathsf{ACE.GenEK}(sk, \mathbb{1}_{pl^*})$. Thus, the result follows by the pseudorandom ciphertext security of ACE and the pseudorandom puncturing point property of $\mathcal{F}$. □

Lemma 31. $\mathsf{Hyb}_2 \approx \mathsf{Hyb}_3$.

Proof. By the punctured key correctness of the cryptographic primitive $\mathcal{F}$ and the punctured key correctness of the scheme PRF, the circuits $P^{(2,1)}$ and $P^{(3,1)}$ have exactly the same functionality, with overwhelming probability. Similarly for $P^{(2,2)}$ and $P^{(3,2)}$. Thus, the result follows by the security of $i\mathcal{O}$. □

Lemma 32. $\mathsf{Hyb}_3 \approx \mathsf{Hyb}_4$.

Proof. Observe that the experiments only uses the PRG output $ct_1^* = G(r_1^*)$ and $ct_2^* = G(r_2^*)$, and the random strings r_1^*, r_2^* do not appear anywhere else. Thus, the result follows by the security of G. □

Lemma 33. $\mathsf{Hyb}_4 \approx \mathsf{Hyb}_5$.

Proof. First, observe that the experiments only use the punctured PRF key $K\{act^*\}$. Thus, by the punctured key security of PRF, we can always change $\mathsf{PRF.Eval}(K, act^*)$ to a uniformly random string. This completes the proof of the case $c' = 1$. When $c' = 2$, observe that the experiment does not use $\mathsf{PRF.Eval}(K, act^*)$, thus we can replace $s_2^* = C^*(act^*) \oplus \mathsf{PRF.Eval}(K, act^*)$ with a uniformly random string. □

Lemma 34. $\mathsf{Hyb}_5 \approx \mathsf{Hyb}_6$.

Proof. This follows directly by the weak resampling property of $\mathcal{F}$. □

Lemma 35. $\mathsf{Hyb}_6 \equiv \mathsf{Hyb}_7$.

Proof. The only difference between Hyb_6 and Hyb_7 is reordering of some independent steps, hence the result follows. □

10 Acknowledgments

We thank Fuyuki Kitagawa and Takashi Yamakawa for sharing [KY25] with us.

References

[Aar09] Scott Aaronson. Quantum copy-protection and quantum money. In *2009 24th Annual IEEE Conference on Computational Complexity*, pages 229–242, 2009.

[Aar16] Scott Aaronson. The complexity of quantum states and transformations: From quantum money to black holes, 2016.

[AB24] Prabhanjan Ananth and Amit Behera. A modular approach to unclonable cryptography. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 3–37, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.

[ACFQ22] Prabhanjan Ananth, Kai-Min Chung, Xiong Fan, and Luowen Qian. Collusion-resistant functional encryption for RAMs. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 160–194, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.

[AL21] Prabhanjan Ananth and Rolando L. La Placa. Secure software leasing. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 501–530, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.

[ALL+21] Scott Aaronson, Jiahui Liu, Qipeng Liu, Mark Zhandry, and Ruizhe Zhang. New approaches for quantum copy-protection. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 526–555, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

[BB84] G Brassard and Charles H Bennett. Quantum cryptography: Public key distribution and coin tossing. In *International conference on computers, systems and signal processing*, pages 175–179, 1984.

[BBV24] James Bartusek, Zvika Brakerski, and Vinod Vaikuntanathan. Quantum state obfuscation from classical oracles. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *56th Annual ACM Symposium on Theory of Computing*, pages 1009–1017, Vancouver, BC, Canada, June 24–28, 2024. ACM Press.

[BKW17] Dan Boneh, Sam Kim, and David J. Wu. Constrained keys for invertible pseudorandom functions. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 237–263, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.

[BV15] Zvika Brakerski and Vinod Vaikuntanathan. Constrained key-homomorphic PRFs from standard lattice assumptions - or: How to secretly embed a circuit in your PRF. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015: 12th Theory of Cryptography Conference, Part II*, volume 9015 of *Lecture Notes in Computer Science*, pages 1–30, Warsaw, Poland, March 23–25, 2015. Springer Berlin Heidelberg, Germany.

[ÇG24a] Alper Çakan and Vipul Goyal. Unclonable cryptography with unbounded collusions and impossibility of hyperefficient shadow tomography. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part III*, volume 15366 of *Lecture Notes in Computer Science*, pages 225–256, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland.

[CG24b] Andrea Coladangelo and Sam Gunn. How to use quantum indistinguishability obfuscation. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, STOC 2024, page 1003–1008, New York, NY, USA, 2024. Association for Computing Machinery.

[ÇGLZR24] Alper Çakan, Vipul Goyal, Chen-Da Liu-Zhang, and João Ribeiro. Unbounded leakage-resilience and intrusion-detection in a quantum world. In Elette Boyle and Mohammad

Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part II*, volume 15365 of *Lecture Notes in Computer Science*, pages 159–191, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland.

[ÇGS25] Alper Çakan, Vipul Goyal, and Omri Shmueli. Public-key quantum fire and key-fire from classical oracles. *arXiv preprint arXiv:2504.16407*, 2025.

[CGY24] Alper Cakan, Vipul Goyal, and Takashi Yamakawa. Anonymous public-key quantum money and quantum voting. *arXiv preprint arXiv:2411.04482*, 2024.

[CHJV15] Ran Canetti, Justin Holmgren, Abhishek Jain, and Vinod Vaikuntanathan. Succinct garbling and indistinguishability obfuscation for RAM programs. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th Annual ACM Symposium on Theory of Computing*, pages 429–437, Portland, OR, USA, June 14–17, 2015. ACM Press.

[CLLZ21] Andrea Coladangelo, Jiahui Liu, Qipeng Liu, and Mark Zhandry. Hidden cosets and applications to unclonable cryptography. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 556–584, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

[DPVR12] Anindya De, Christopher Portmann, Thomas Vidick, and Renato Renner. Trevisan's extractor in the presence of quantum side information. *SIAM Journal on Computing*, 41(4):915–940, 2012.

[GGG+14] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 578–602, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.

[GGH+13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th Annual Symposium on Foundations of Computer Science*, pages 40–49, Berkeley, CA, USA, October 26–29, 2013. IEEE Computer Society Press.

[GJKS15] Vipul Goyal, Abhishek Jain, Venkata Koppula, and Amit Sahai. Functional encryption for randomized functionalities. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015: 12th Theory of Cryptography Conference, Part II*, volume 9015 of *Lecture Notes in Computer Science*, pages 325–351, Warsaw, Poland, March 23–25, 2015. Springer Berlin Heidelberg, Germany.

[KK10] Roy Kasher and Julia Kempe. Two-source extractors secure against quantum adversaries. In *International Workshop on Randomization and Approximation Techniques in Computer Science*, pages 656–669. Springer, 2010.

[KY25] Fuyuki Kitagawa and Takashi Yamakawa. Foundations of single-decryptor encryption. 2025.

[LLQZ22] Jiahui Liu, Qipeng Liu, Luowen Qian, and Mark Zhandry. Collusion resistant copy-protection for watermarkable functionalities. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part I*, volume 13747 of

Lecture Notes in Computer Science, pages 294–323, Chicago, IL, USA, November 7–10, 2022. Springer, Cham, Switzerland.

[LMW23] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. In Barna Saha and Rocco A. Servedio, editors, *55th Annual ACM Symposium on Theory of Computing*, pages 595–608, Orlando, FL, USA, June 20–23, 2023. ACM Press.

[NC10] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[RRV99] Ran Raz, Omer Reingold, and Salil Vadhan. Error reduction for extractors. In *40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039)*, pages 191–201. IEEE, 1999.

[SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th Annual ACM Symposium on Theory of Computing*, pages 475–484, New York, NY, USA, May 31 – June 3, 2014. ACM Press.

[Wie83] Stephen Wiesner. Conjugate coding. *SIGACT News*, 15(1):78–88, jan 1983.

[Zha12] Mark Zhandry. How to construct quantum random functions. In *53rd Annual Symposium on Foundations of Computer Science*, pages 679–687, New Brunswick, NJ, USA, October 20–23, 2012. IEEE Computer Society Press.

[Zha20] Mark Zhandry. Schrödinger's pirate: How to trace a quantum decoder. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part III*, volume 12552 of *Lecture Notes in Computer Science*, pages 61–91, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland.